Programmation réseau en Java

Jonathan Lejeune

Sorbonne Université/LIP6

SRCS – Master 1 SAR 2023/2024

sources :

Développons en Java, Jean-Michel Doudoux

Cours précédents de J. Sopena

La communication en réseau

Quelles sont les abstractions réseaux nécessaires pour faire fonctionner une application répartie ?

Un modèle standard en 7 couches

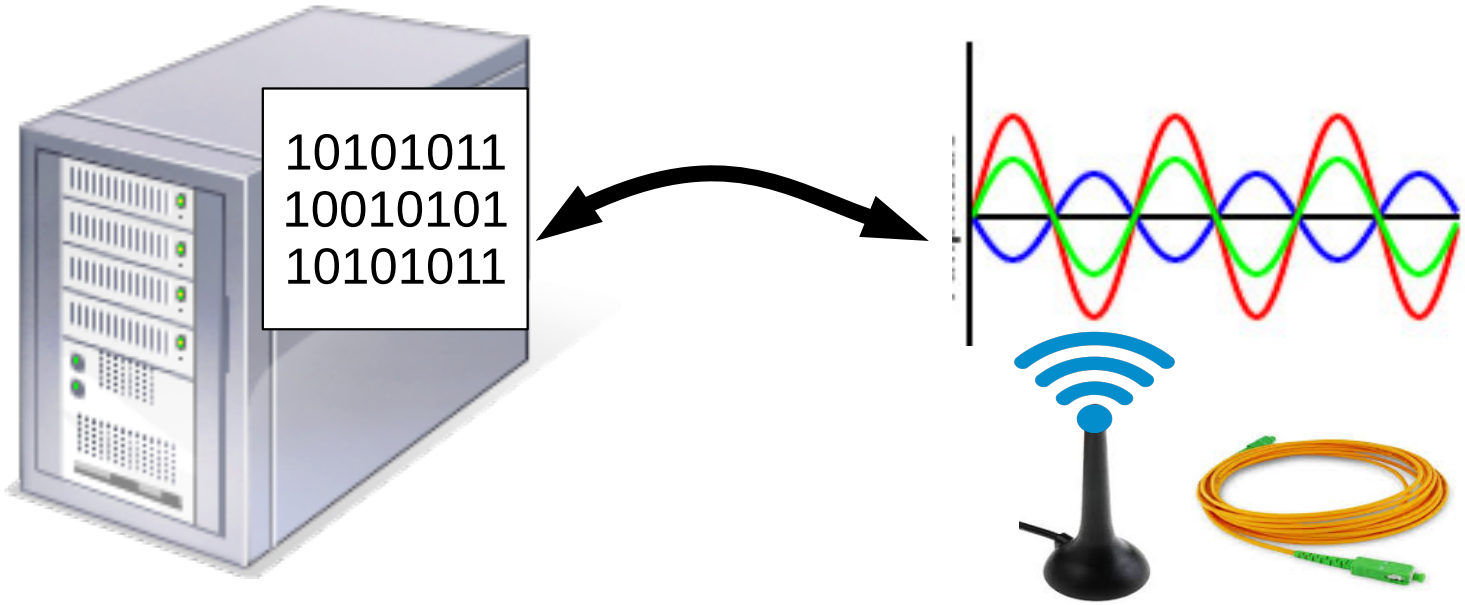
Le modèle OSI :

- **Couche 7 - Applicative** : Les applications utilisant le réseau
- **Couche 6 - Présentation** : Représentation des données
- **Couche 5 - Session** : Établissement et maintien des sessions
- **Couche 4 - Transport** : Liaison de bout en bout
- **Couche 3 - Réseau** : Adressage et routage entre machines
- **Couche 2 - Liaison** : Communication point à point
- **Couche 1 - Physique** : Support de transmission

Le modèle OSI : la couche physique (niv. 1)

Caractéristiques

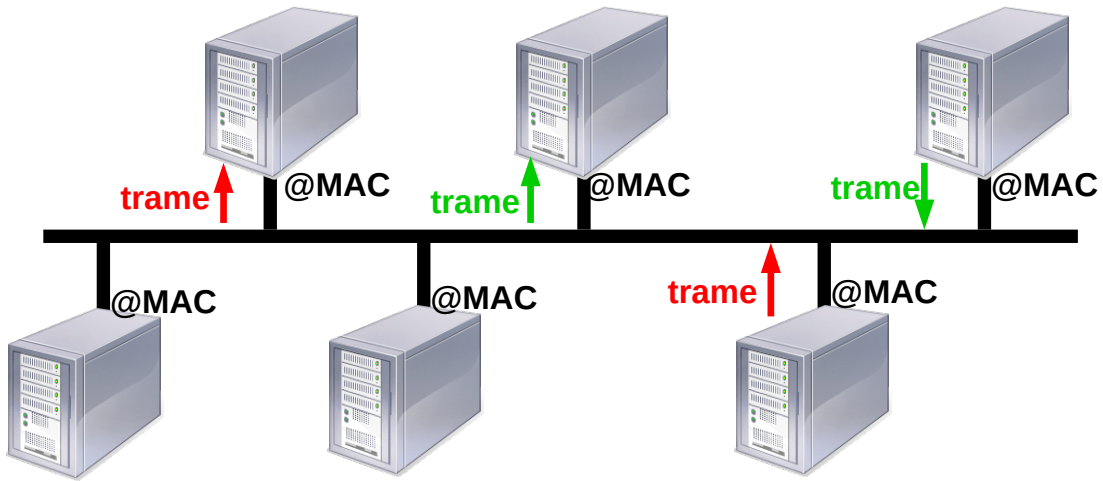
- **Objectif :** encoder/décoder des bits vers/depuis un signal d'un support de transmission (onde, fibre, câble)
- **Protocol Data Unit :** bit
- **Exemples de protocole :** BHDn, MLT-3, HDB3



Le modèle OSI : la couche liaison (niv. 2)

Caractéristiques

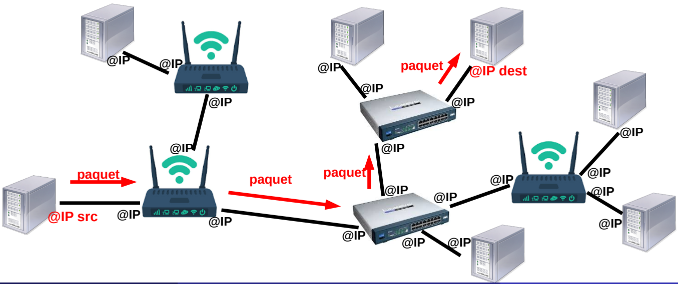
- **Objectif** : Envoyer/recevoir des messages entre des machines reliées physiquement par un support (point à point)
- **Protocol Data Unit** : trame
- **Exemples de protocole** : Ethernet, ATM, PPP
- **Services offerts :**
  - Contrôle d'erreur : détection bit flip
  - Contrôle d'accès au support : adressage physique (MAC), protocole d'accès multiples (CSMA/CD)



Le modèle OSI : la couche réseau (niv. 3)

Caractéristiques

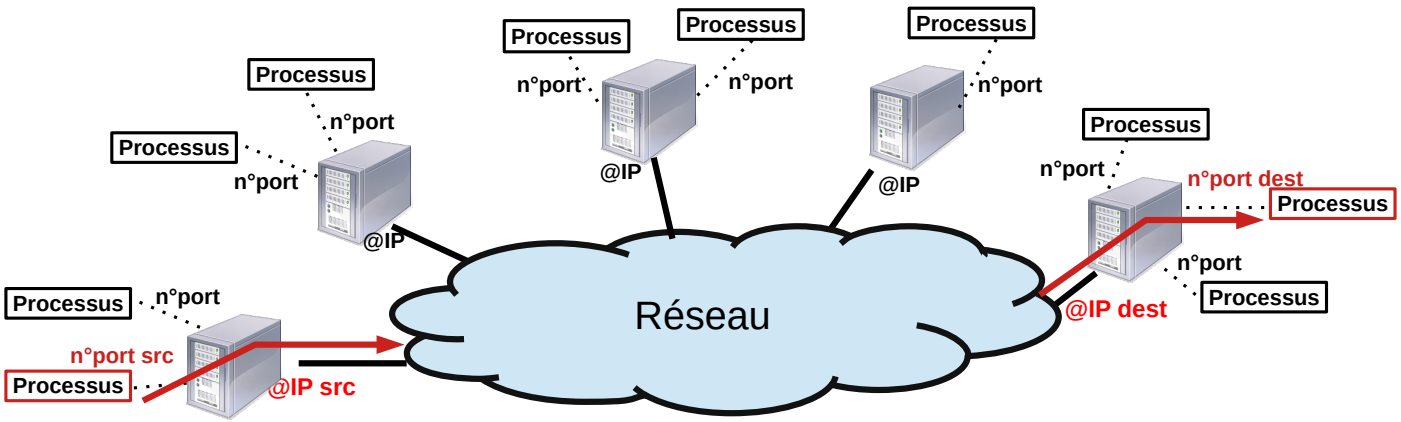
- **Objectif** : Router un message d'une machine source vers une machine destination
- **Protocol Data Unit** : paquet
- **Exemple de protocole** : Internet Protocol
- **Services offerts :**
  - routage : trouver un chemin pour acheminer un message
  - adressage logique (adresse IP)



Le modèle OSI : la couche transport (niv. 4)

Caractéristiques

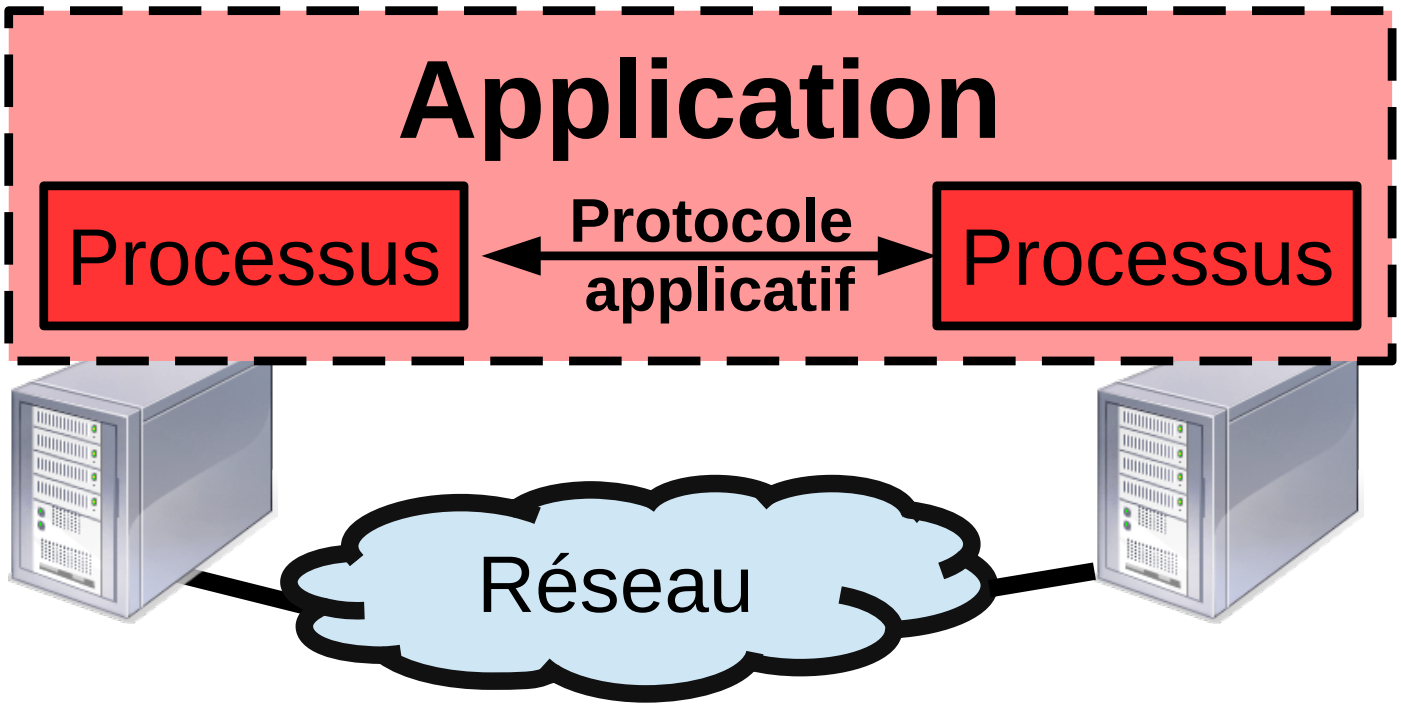
- **Objectif** : Faire communiquer deux processus hébergés sur deux machines distantes
- **Protocol Data Unit** : segment ou datagramme
- **Exemple de protocole** : TCP, UDP
- **Services offerts :**
  - Multiplexage / Démultiplexage : permettre à plusieurs processus de communiquer sur le même réseau $\Rightarrow$ **notion de numéro de ports**
  - Contrôle de l'intégrité des données pour TCP



Le modèle OSI : les couches applicatives (niv. 5 à 7)

Caractéristiques communes

- Couches qui définissent les interactions entre deux processus distants d'une même application répartie.
- **Protocol Data Unit** : donnée ou message applicatif



Le modèle OSI : les couches applicatives (niv. 5 à 7)

Caractéristiques couche session (niv. 5)

- **Objectif :** maintenir des états sur la communication entre deux processus d'une même application
- Exemple de protocole : appel de procédures distantes

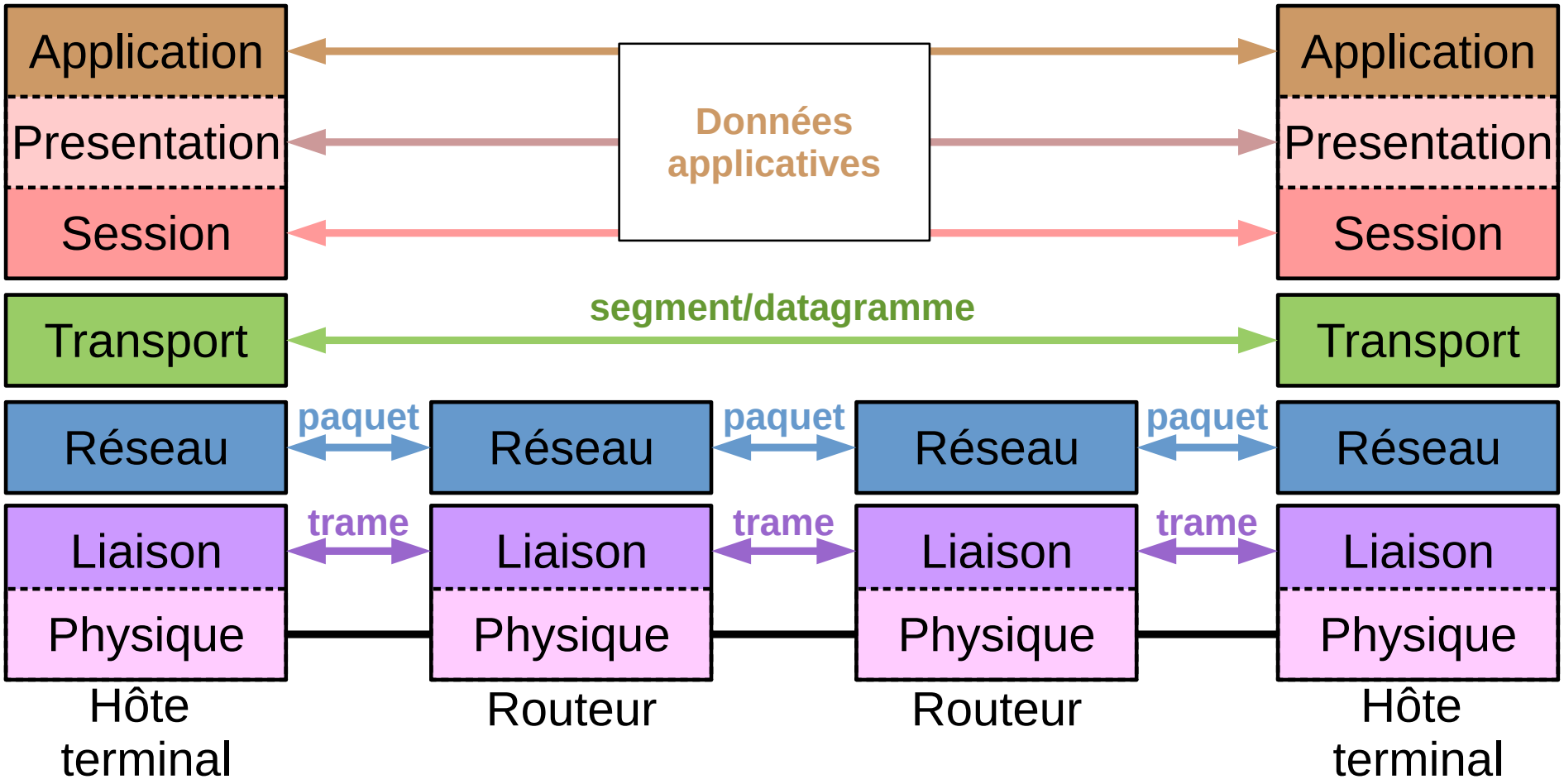
Caractéristiques couche présentation (niv. 6)

- **Objectif :** définir une représentation commune des données entre deux processus d'une même application
- Exemple de protocole : protocole de sérialisation/ déserialisation

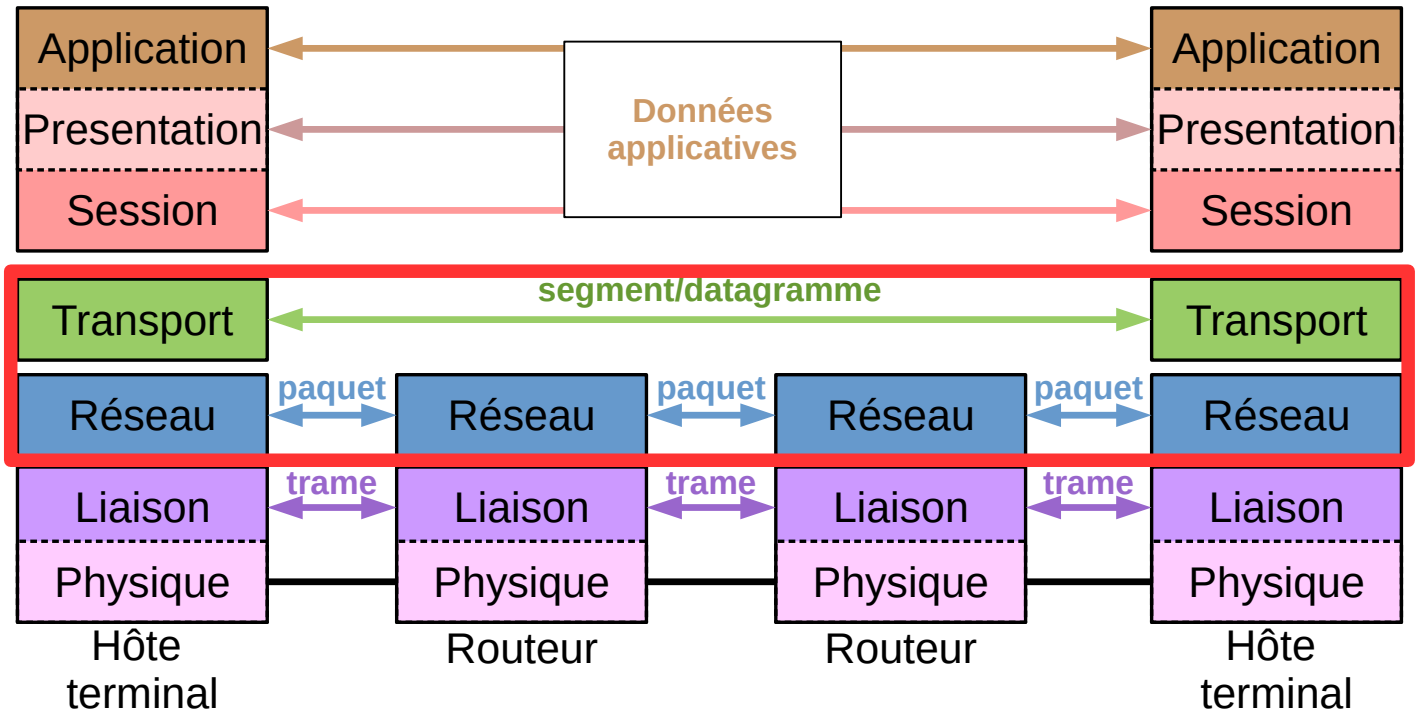
Caractéristiques couche application (niv. 7)

- **Objectif :** définir les interactions entre les composants d'une application répartie
- Exemples de protocole applicatif : HTTP, FTP, SSH, DNS

Synthèse des couches réseaux



Vers la socket



Réponse à la problématique

Pour développer une application répartie nous avons besoin d'un outil qui exploite l'abstraction :

- du niveau 3 pour désigner une machine hôte sur le réseau
- du niveau 4 pour désigner un processus sur une machine hôte

Mode connecté vs. Mode non connecté

Le mode connecté

La communication entre deux machines est **précédée d'une connexion** et **suivie d'une fermeture** :

- ✓ Facilite la gestion d'état
- ✓ Meilleur contrôle des communications (début/fin, détection faute . . .)
- ✗ Seule la communication unicast est possible
- ✗ Plus lent au démarrage

Le mode non connecté

Les messages sont envoyés librement :

- ✓ Plus facile à mettre en œuvre
- ✓ Plus rapide au démarrage
- ✗ Robustesse faible

Les protocoles de transport TCP et UDP

Transmission Control Protocol : communication type *téléphone*

- **Mode connecté :** protocole de prise de connexion
- **Sans perte :** un message arrive au moins une fois
- **Sans duplication :** un message arrive au plus une fois
- **Avec fragmentation :** les messages peuvent être coupés
- **Ordre respecté :** messages délivrés dans l'ordre d'émission

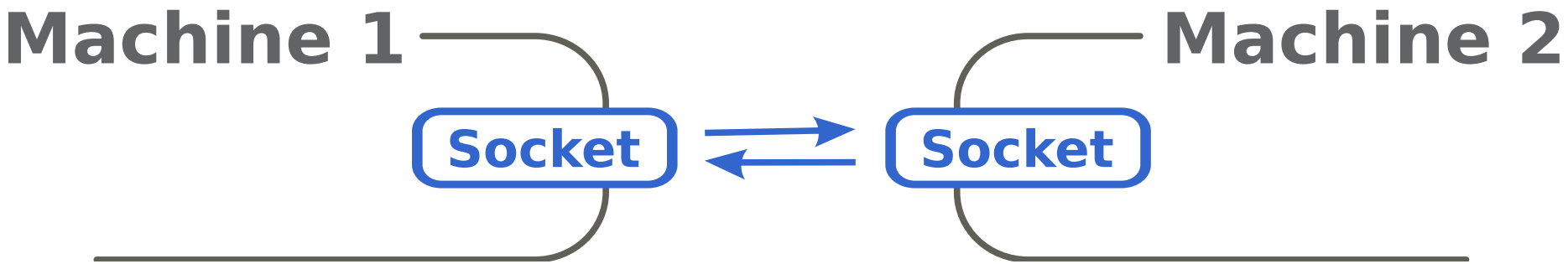
User Datagram Protocol : communication type *courrier*

- **Mode non connecté :** pas de protocole de connexion
- **Avec perte :** l'émetteur n'est pas assuré de la délivrance
- **Avec duplication :** un message peut arriver plus d'une fois
- **Sans fragmentation :** les messages envoyés ne sont jamais coupés
  $\Rightarrow$ soit un message arrive entièrement, soit il n'arrive pas

Les sockets

Définition

API de communication inter-processus (IPC) offerte par le système d'exploitation permettant d'exploiter uniformément les fonctionnalités offertes par les services réseaux.



Identification des sockets

- En absence de connexion : $< @IP, port >$
- En cas de connexion : $(< @IP_{src}, port_{src} >, < @IP_{dest}, port_{dest} >)$
  Attention : Deux sockets peuvent occuper le même port local si elles sont connectées sur des IPs et/ou ports différents.

L'API réseau en java

Les classes de socket

- pour TCP : **ServerSocket** et **Socket**
- pour UDP : **DatagramSocket**

Les classes pour gérer les adresses

- **InetAddress** : représente une adresse internet (ip ou nom de domaine)
- **SocketAddress** : représente l'adresse d'une socket

La classe InetAddress

```
InetAddress me = InetAddress.getByName("sopena.fr");
System.out.println(me.getHostAddress()+" = "+me.getHostName());
```

```
90.46.19.126 = sopena.fr
```

```
InetAddress g = InetAddress.getByAddress(new byte[] {8,8,8,8});
System.out.println(g.getHostAddress()+" = "+g.getCanonicalHostName());
```

```
8.8.8.8 = google-public-dns-a.google.com
```

```
InetAddress ici = InetAddress.getLocalHost();
System.out.println(ici.getHostAddress()+" = "+ici.getHostName());
```

```
132.227.112.199 = ppti-14-508-07.ufr-info-p6.jussieu.fr
```

⇒ Pas de constructeur mais des méthodes statiques

La classe InetSocketAddress

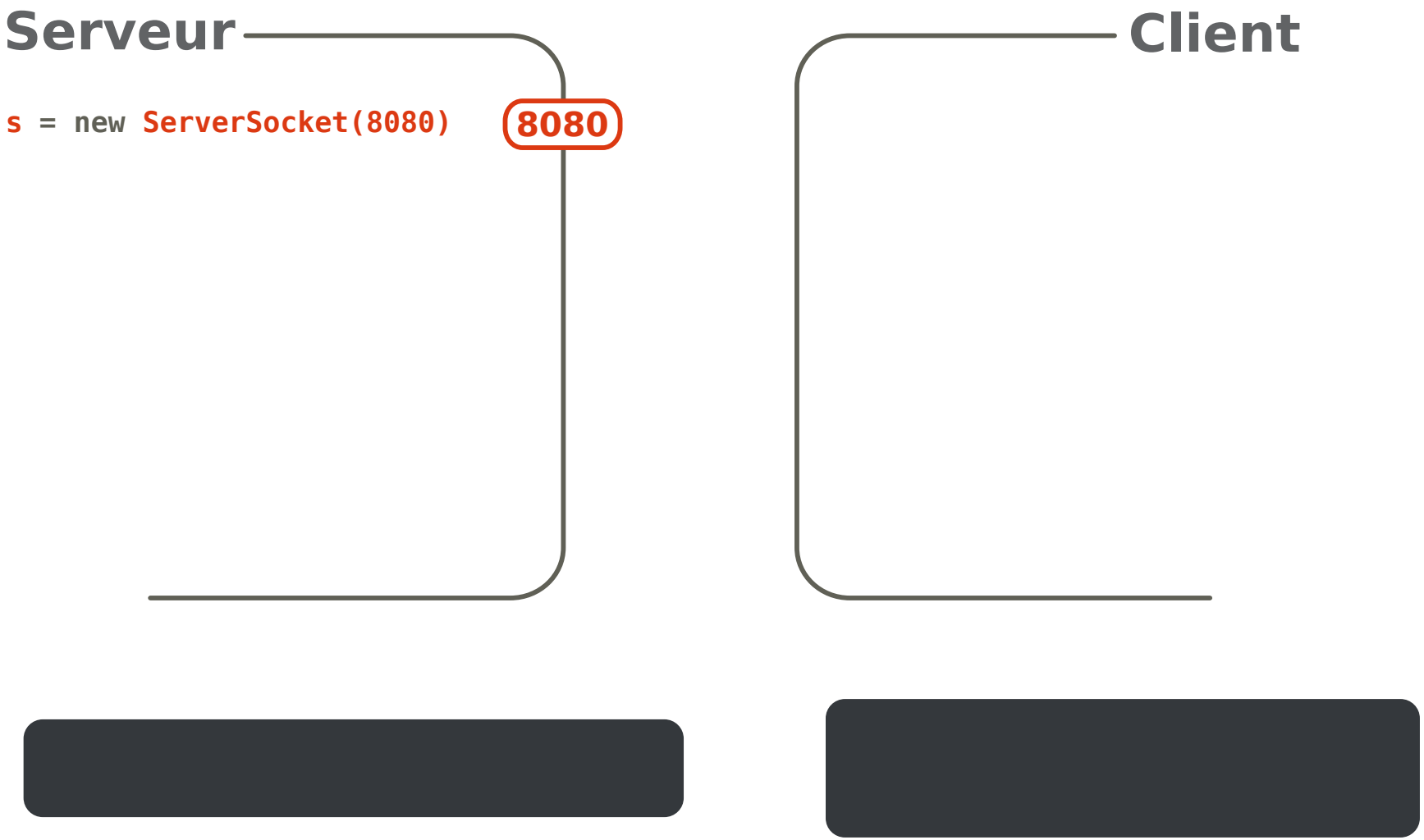
Caractéristiques

- Associe une **adresse IP** et **un numéro de port**
- Hérite de la classe abstraite SocketAddress
- Les instances sont des objets immuables
- Elle n'associe pas le protocole de transport utilisé

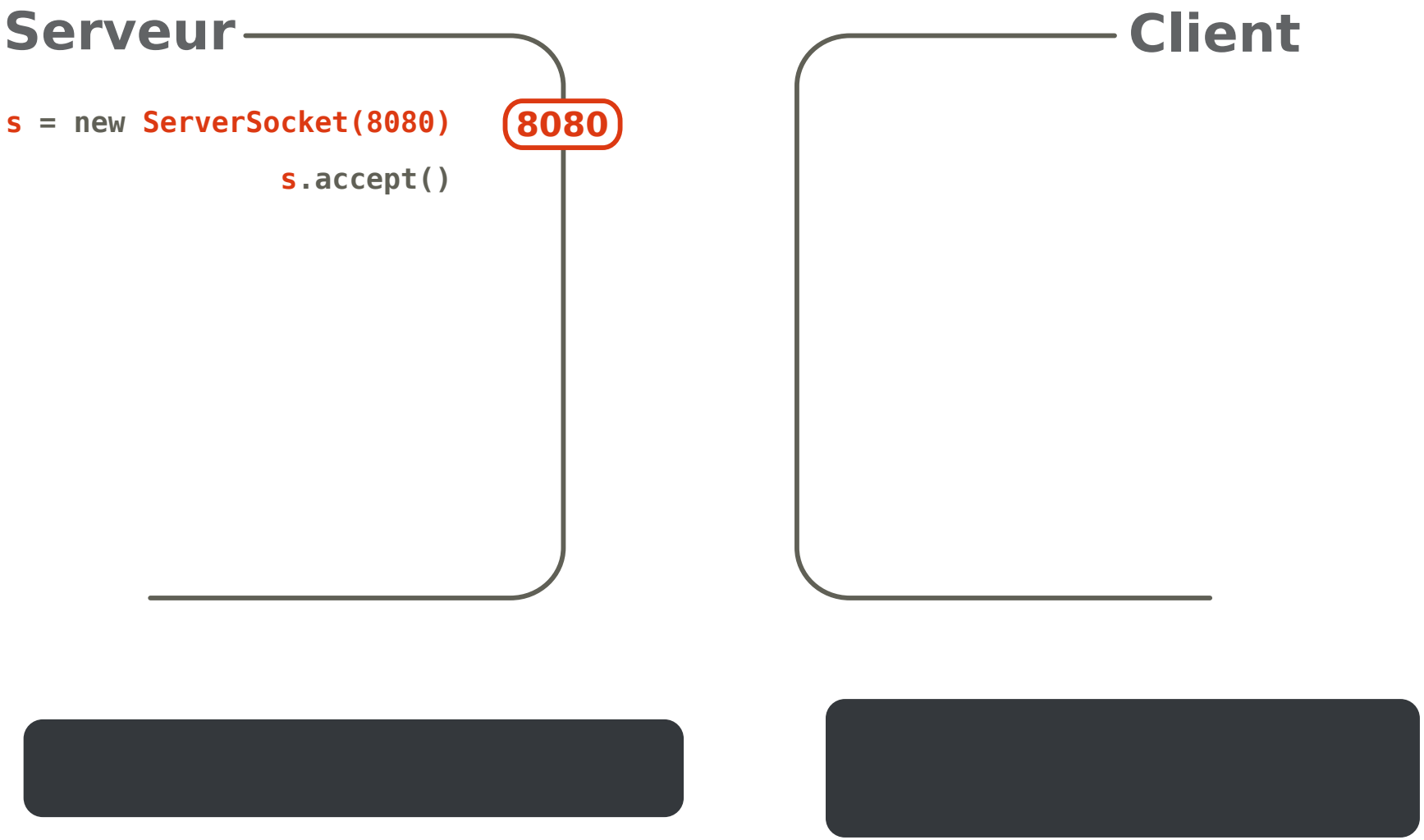
```
//construction d'une nouvelle InetSocketAddress
InetSocketAddress localhost =
                new InetSocketAddress("localhost", 8080);
...
//recupération à partir une socket de connexion
SocketAddress ici = socket.getLocalSocketAddress();
SocketAddress labas = socket.getRemoteSocketAddress();
```

Utilisation d'une socket TCP

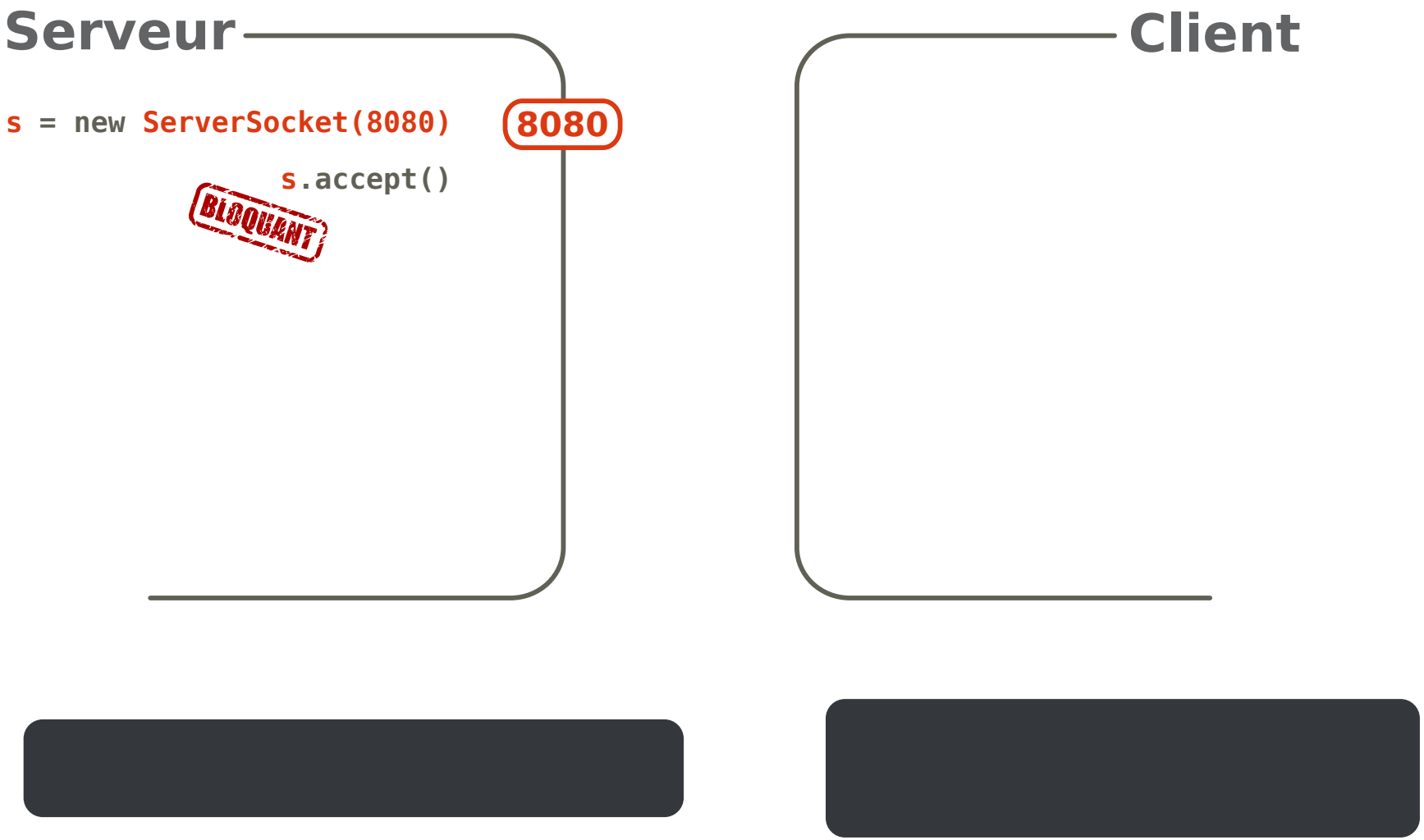
Utilisation d'une socket TCP



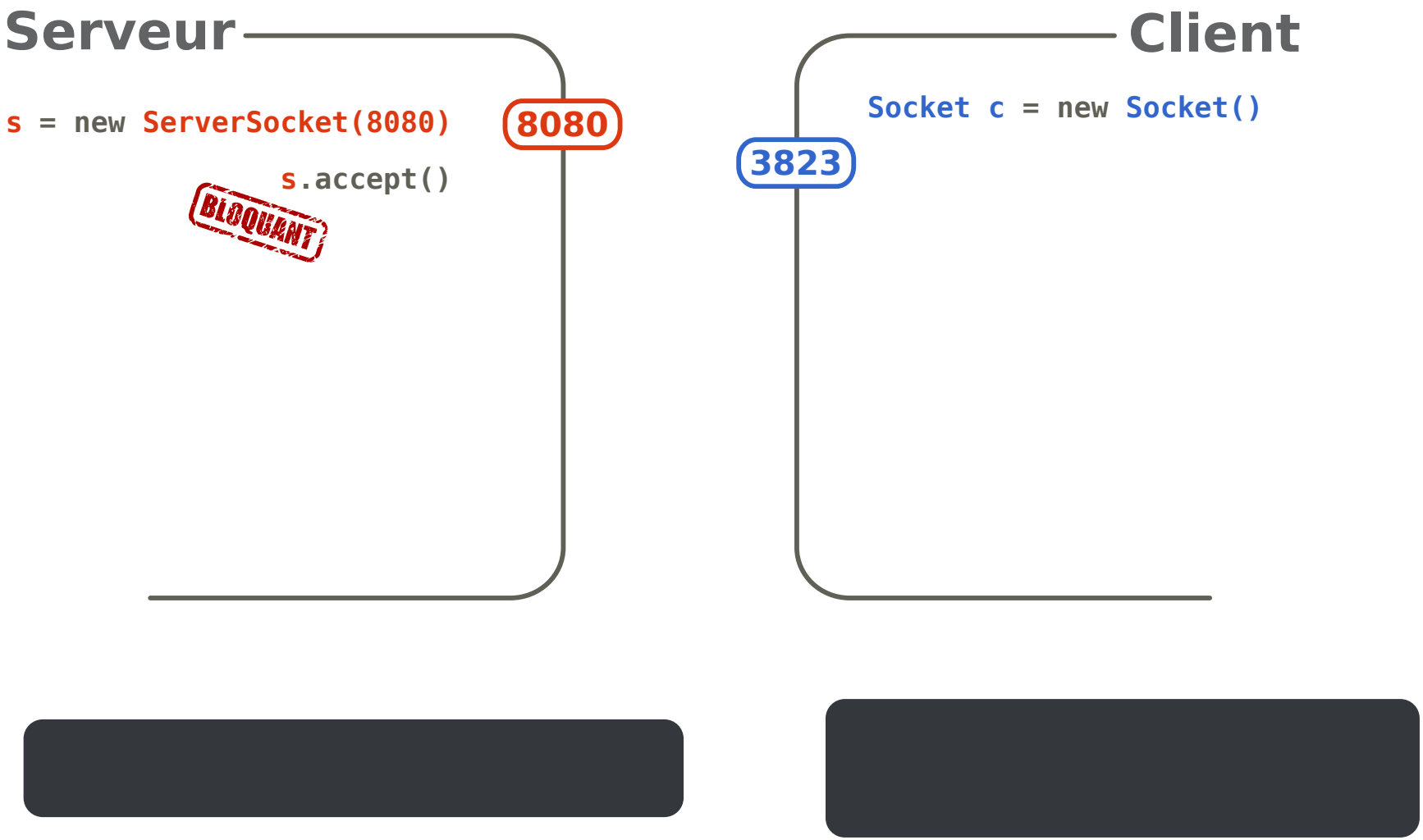
Utilisation d'une socket TCP



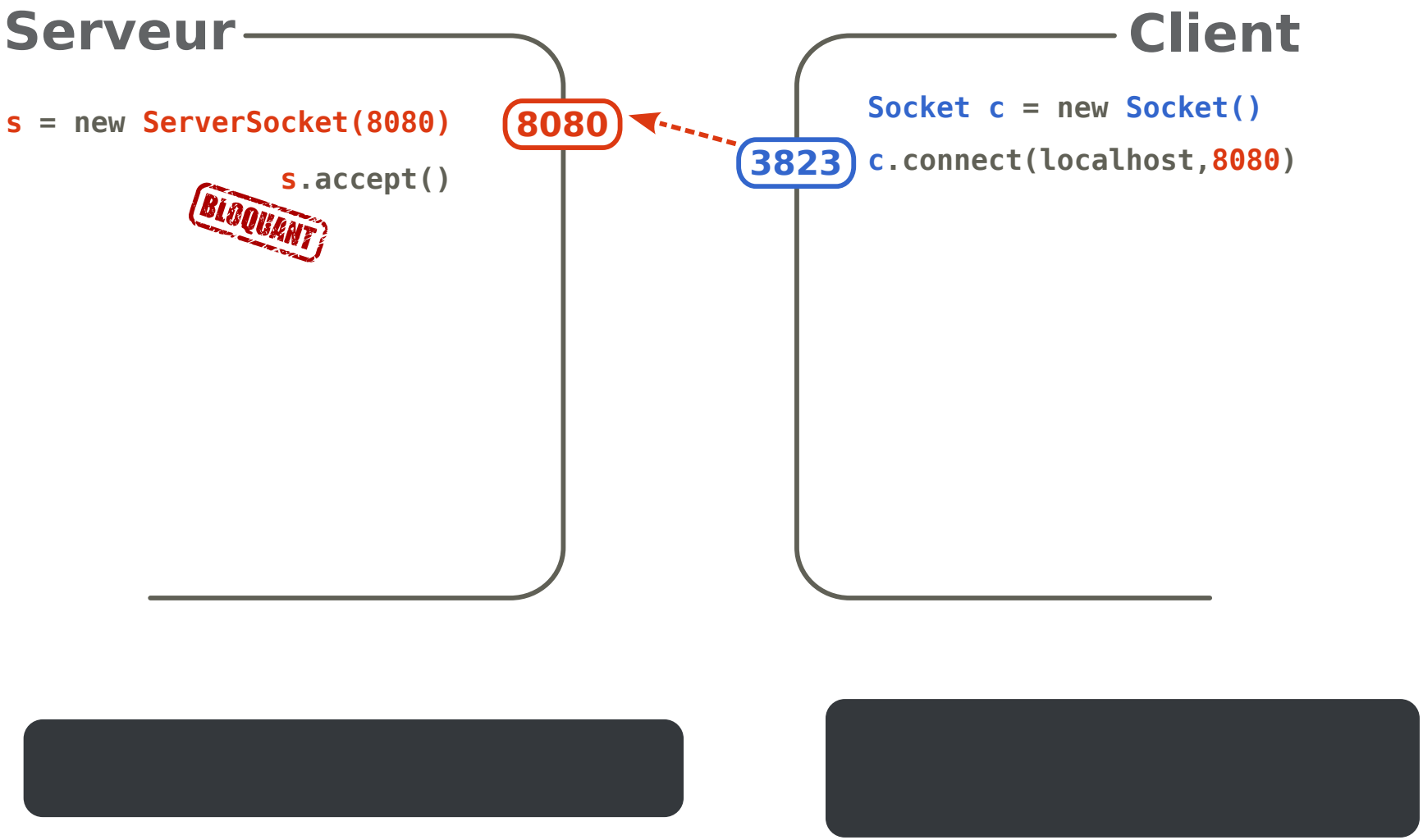
Utilisation d'une socket TCP



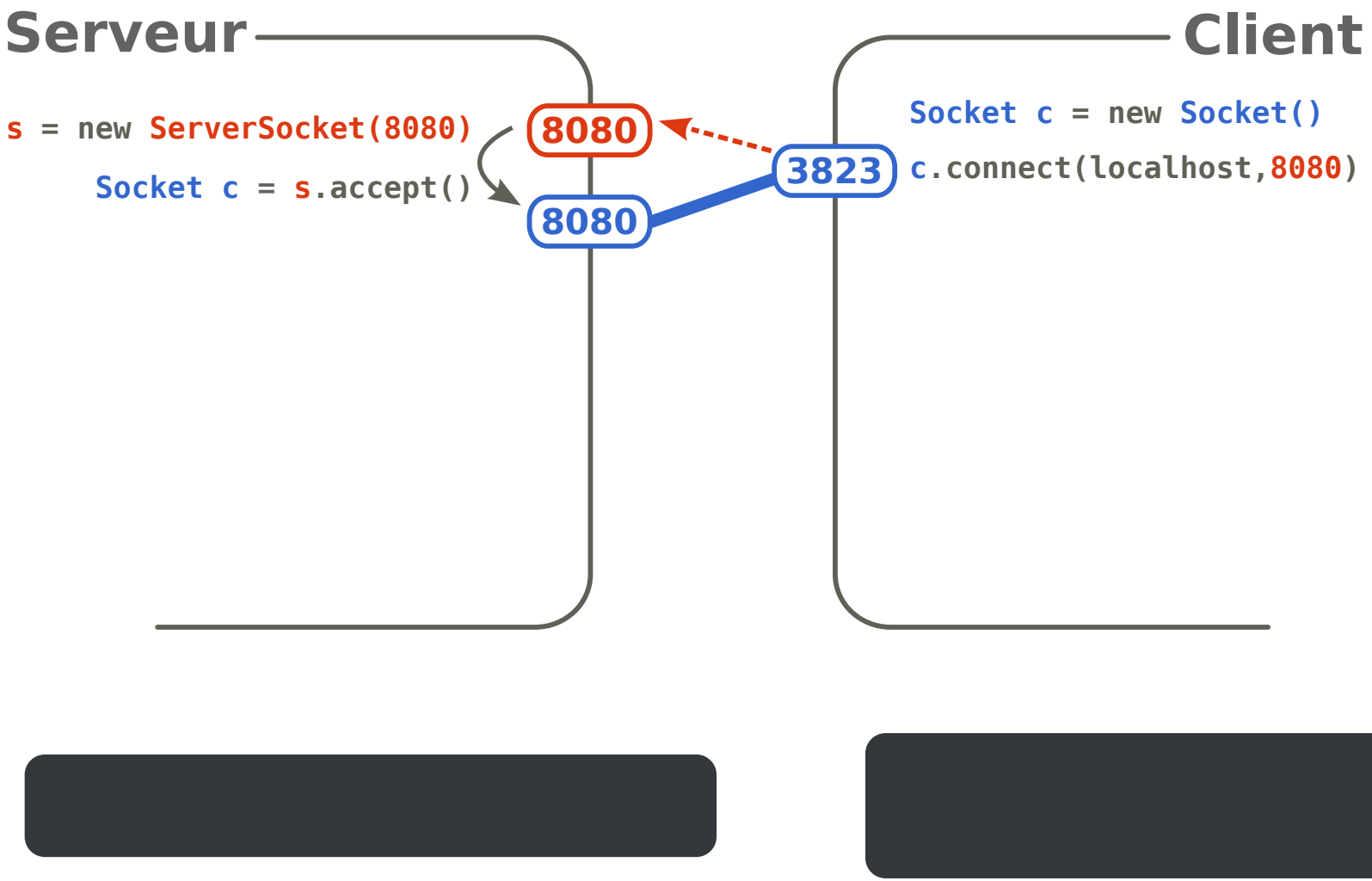
Utilisation d'une socket TCP



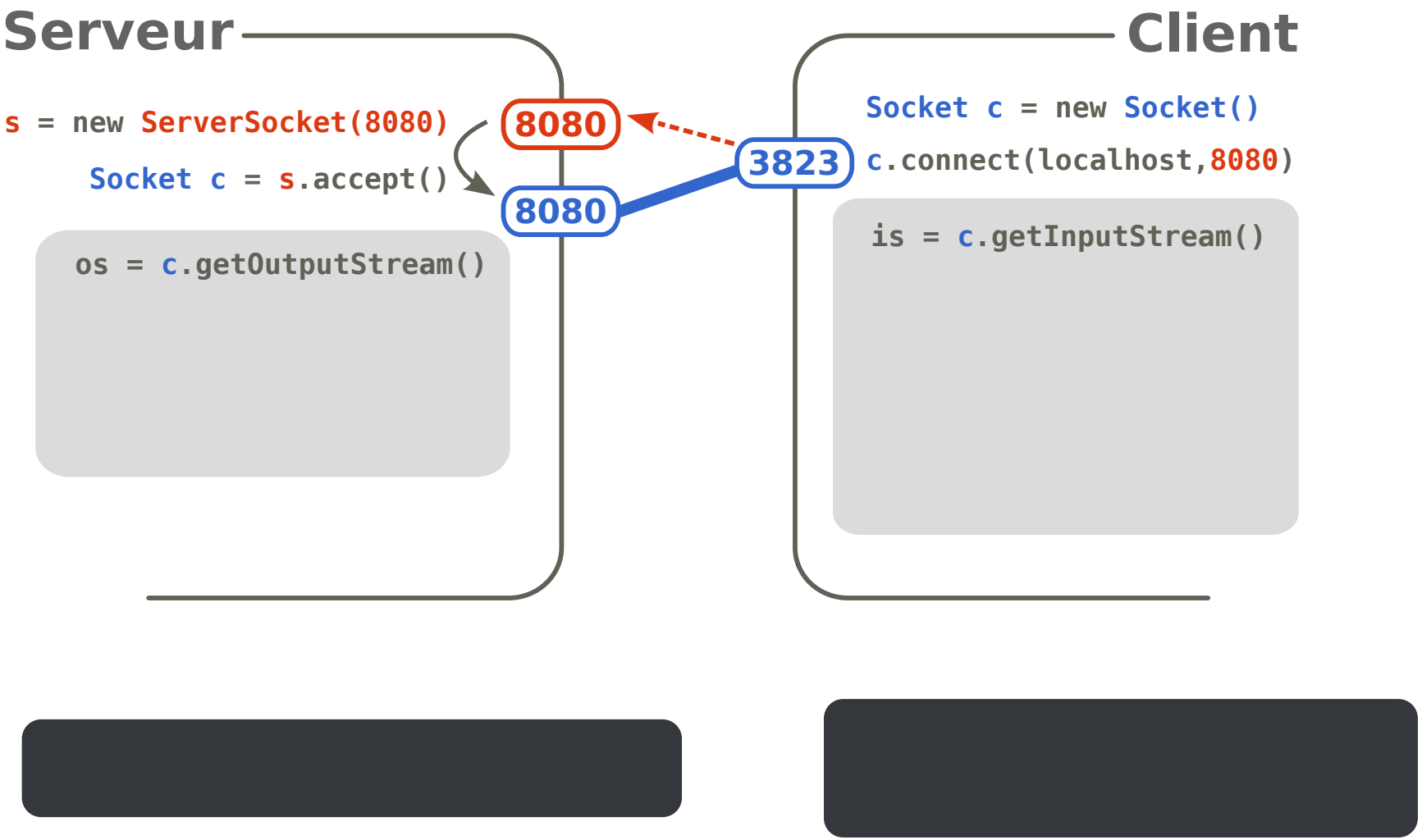
Utilisation d'une socket TCP



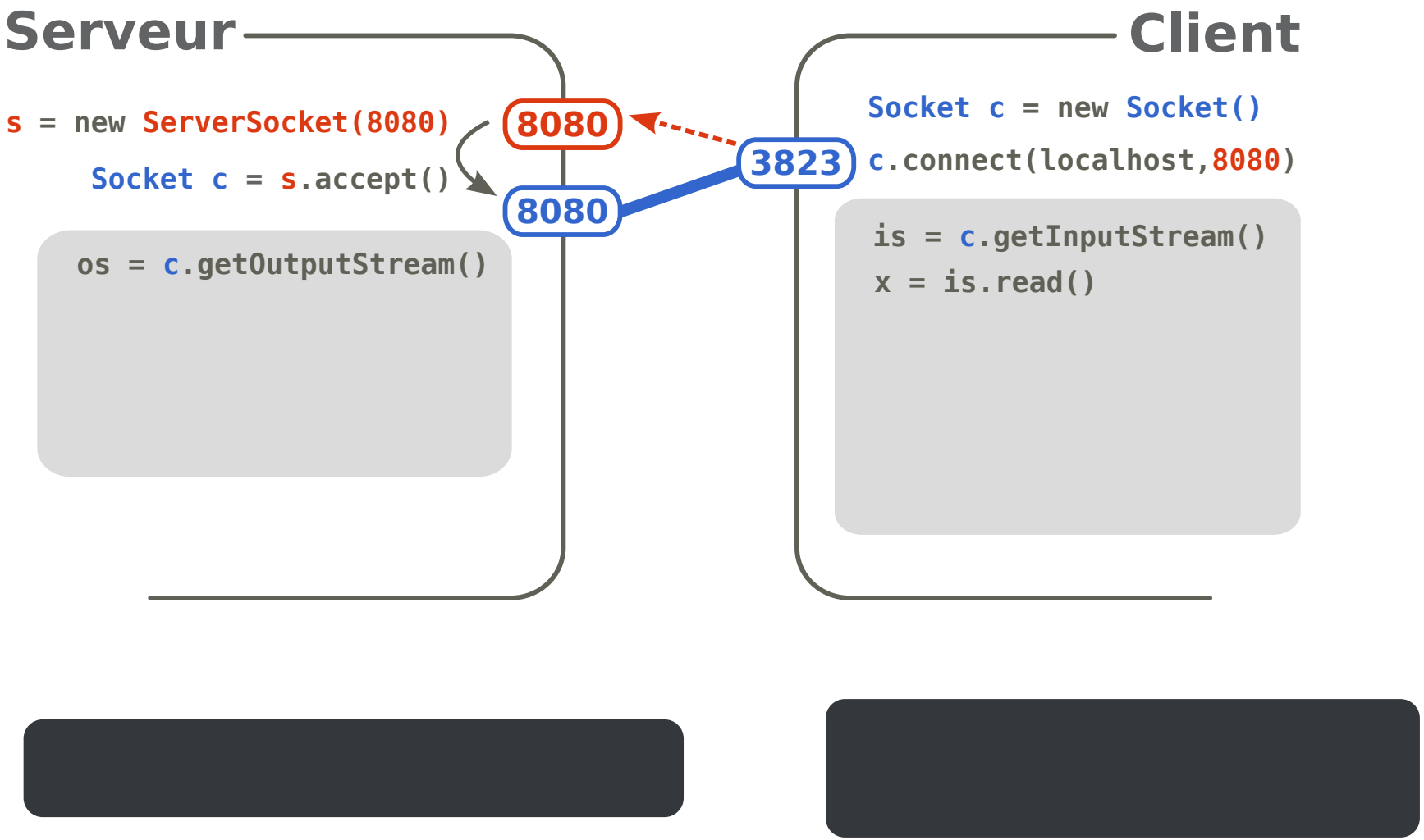
Utilisation d'une socket TCP



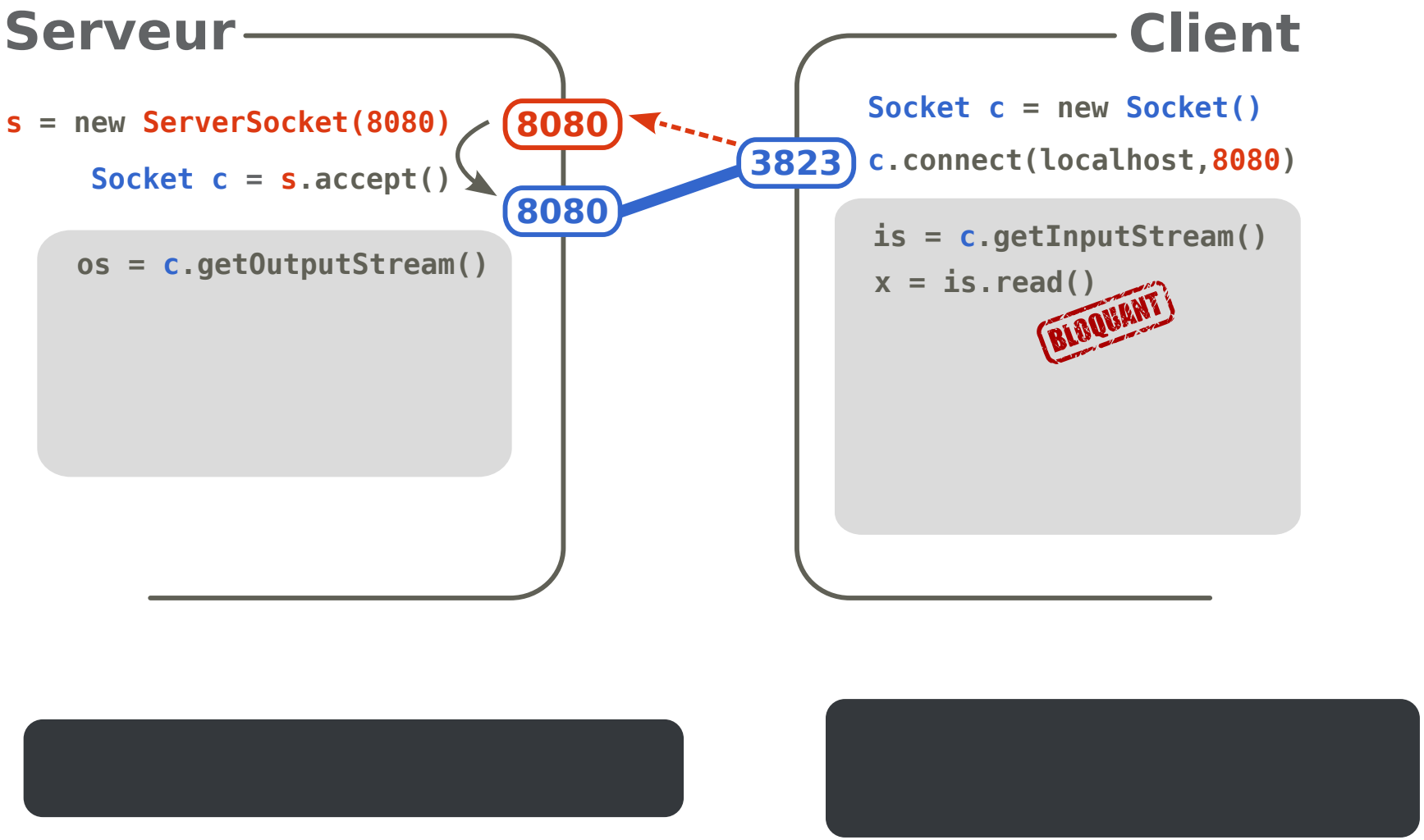
Utilisation d'une socket TCP



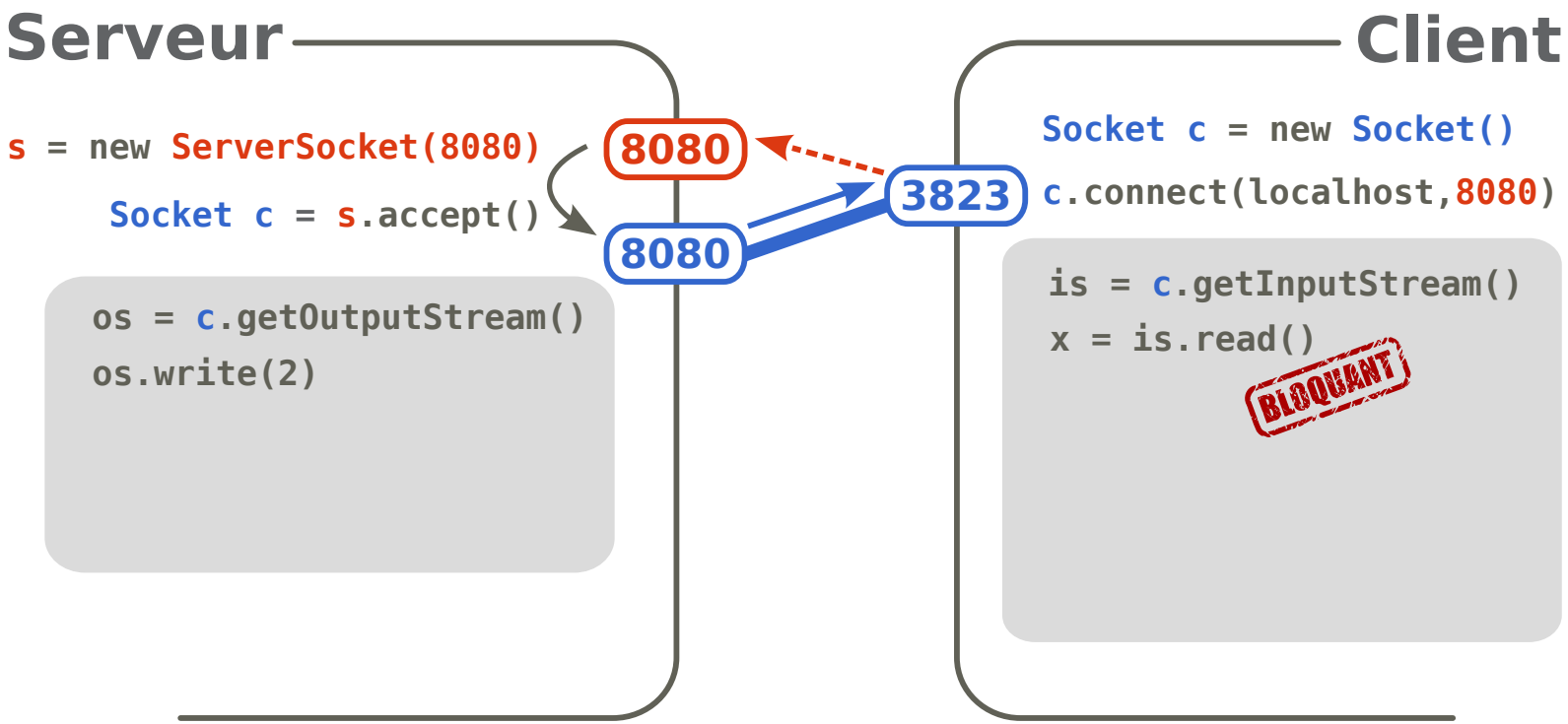
Utilisation d'une socket TCP



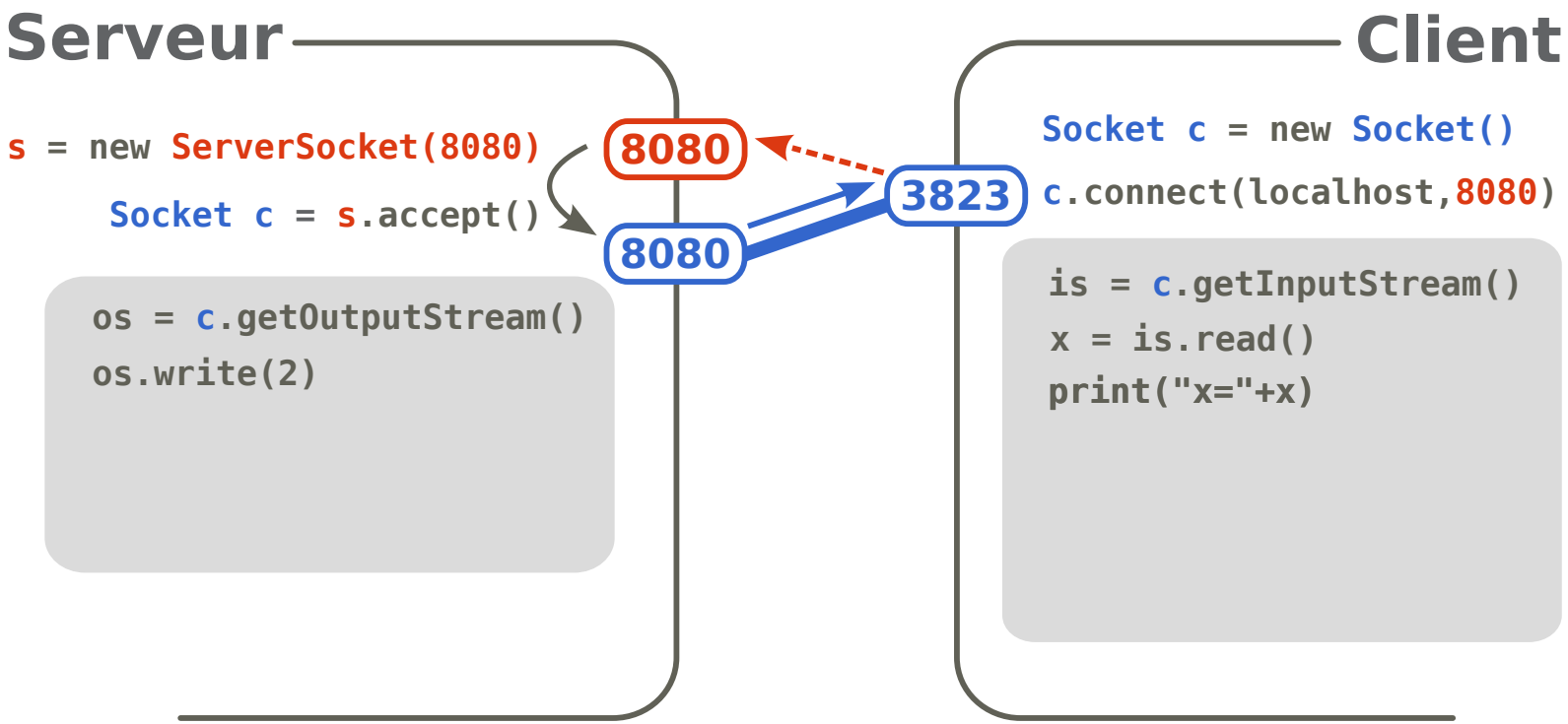
Utilisation d'une socket TCP



Utilisation d'une socket TCP

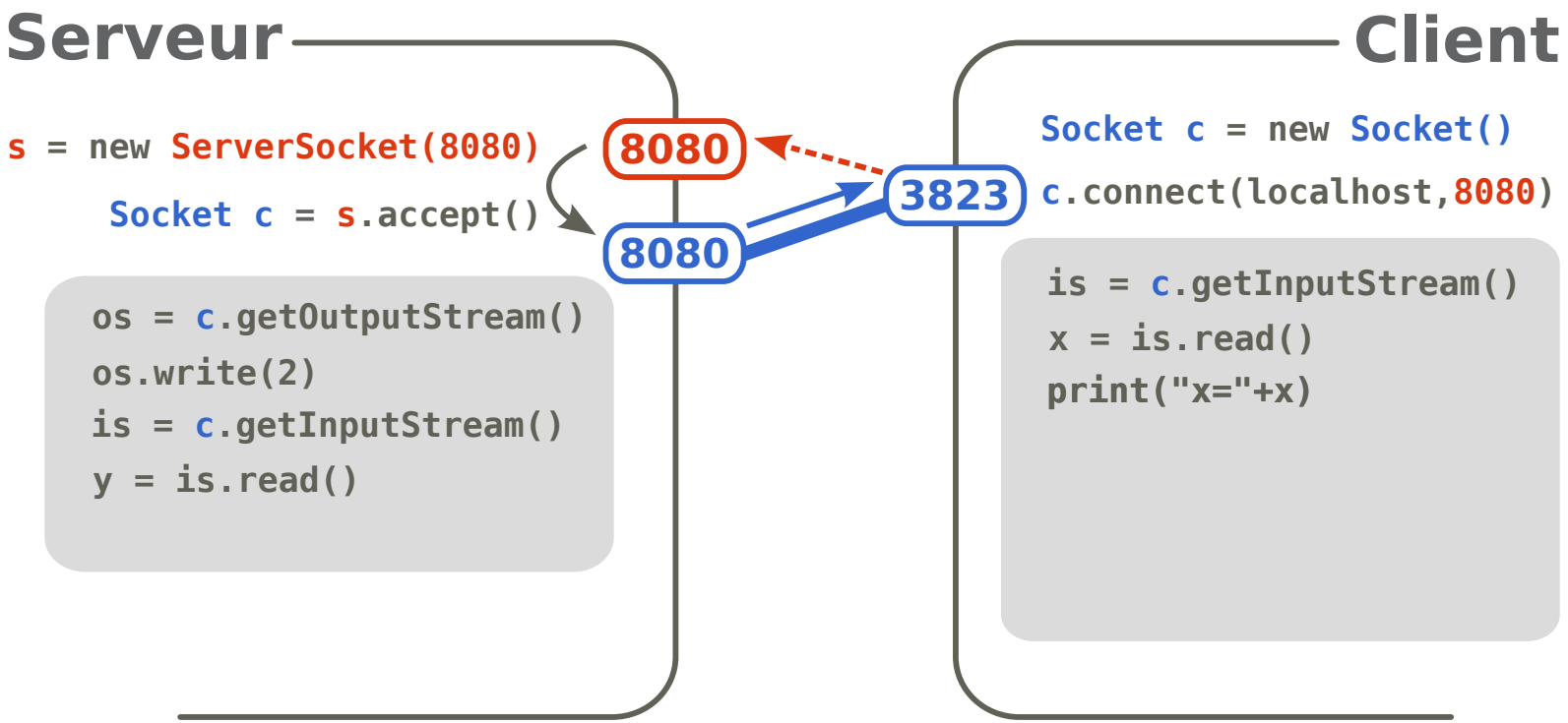


Utilisation d'une socket TCP



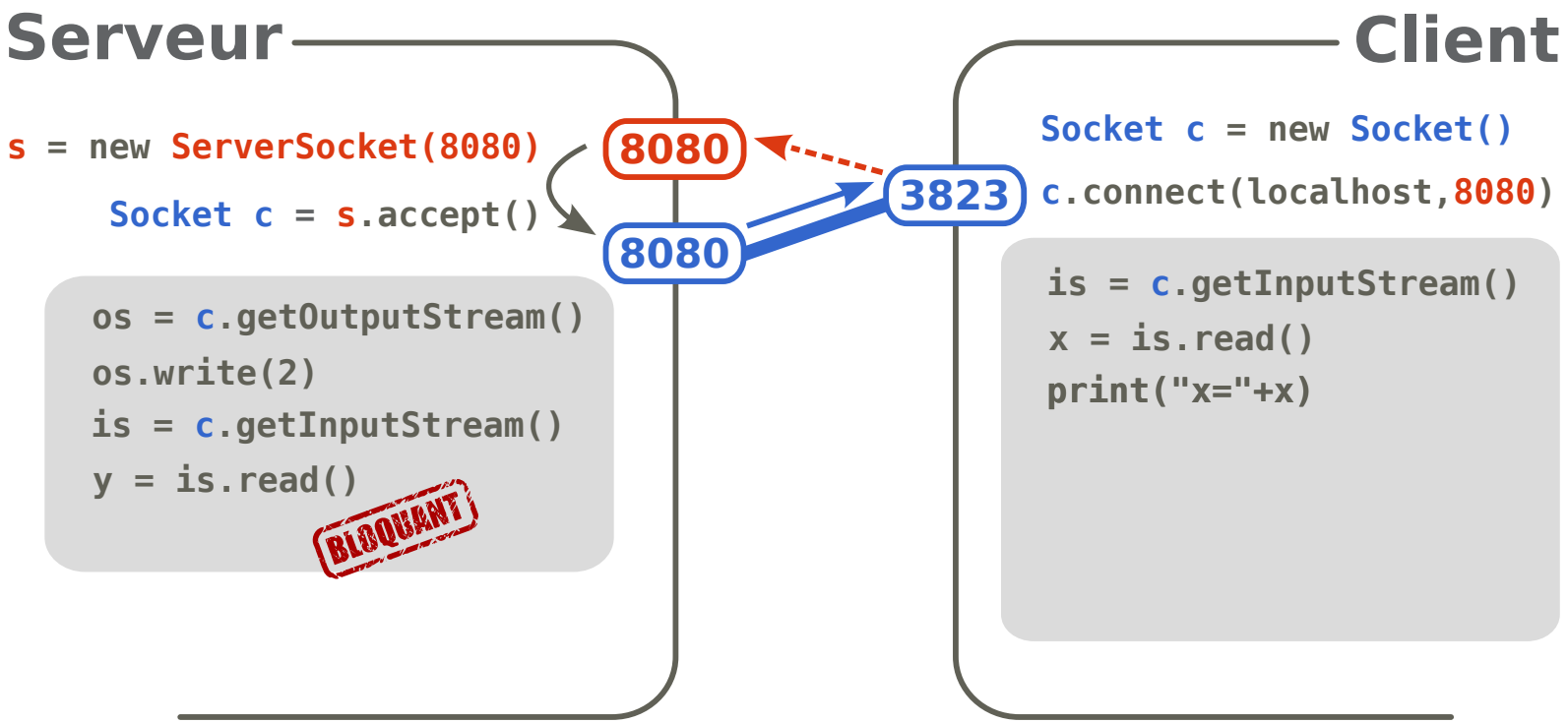
x=2

Utilisation d'une socket TCP



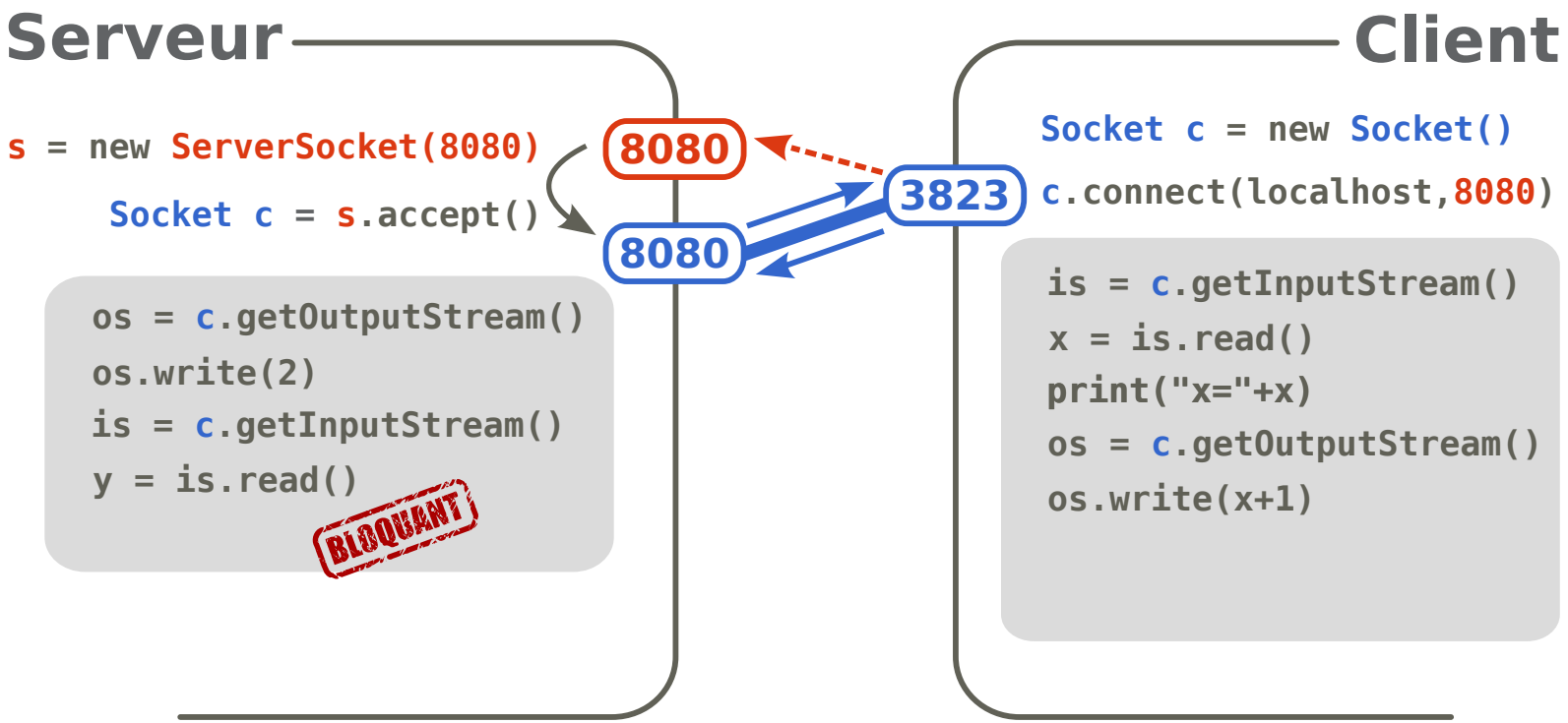
x=2

Utilisation d'une socket TCP



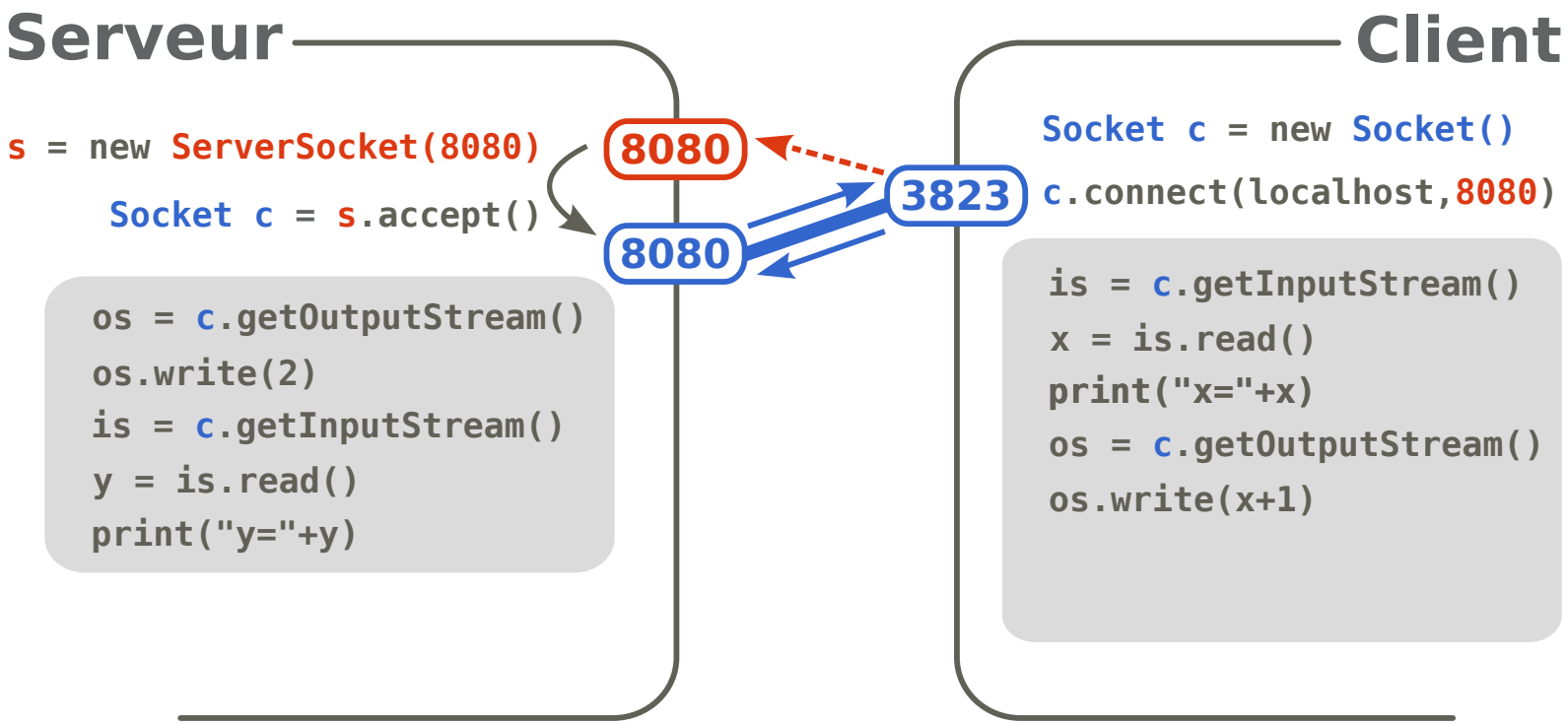
x=2

Utilisation d'une socket TCP



x=2

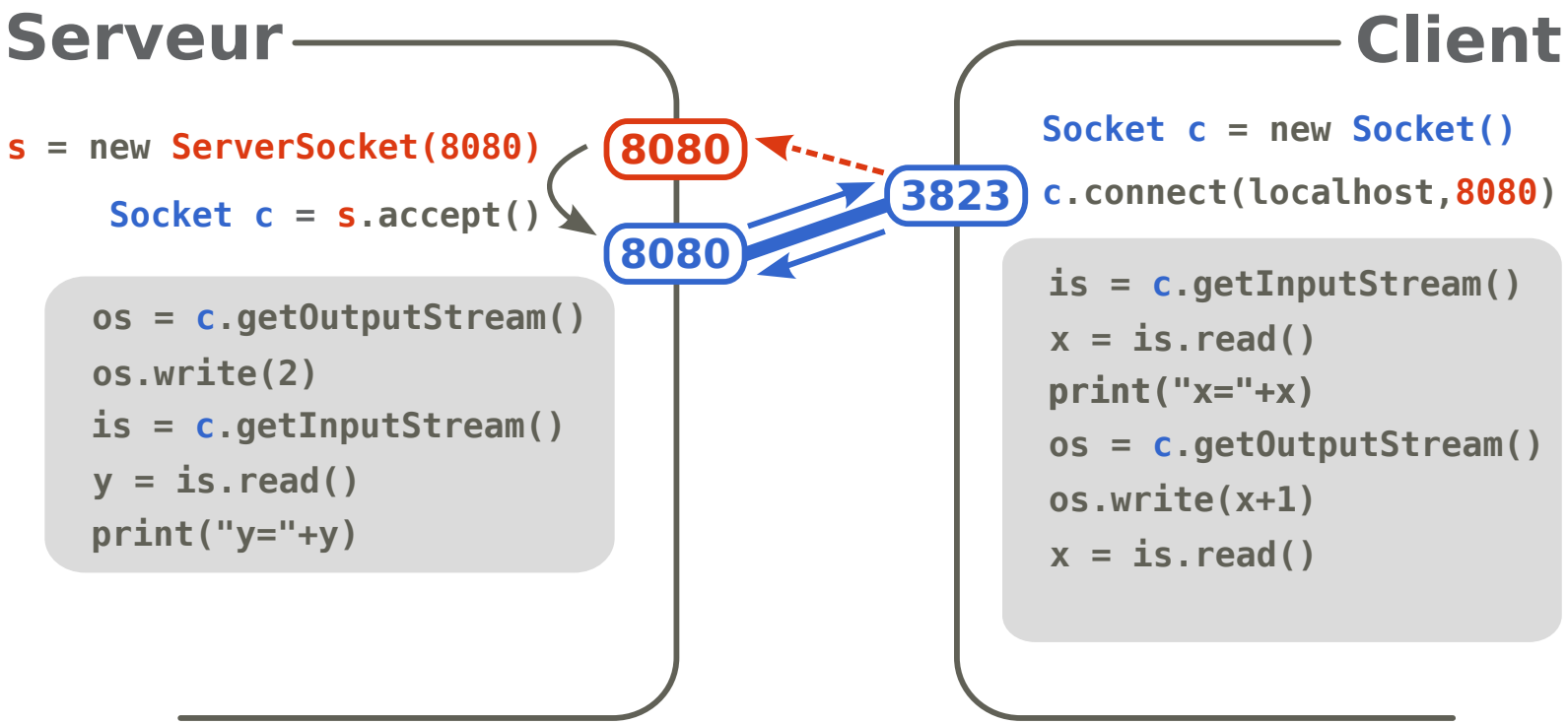
Utilisation d'une socket TCP




```
y=3
```

```
x=2
```

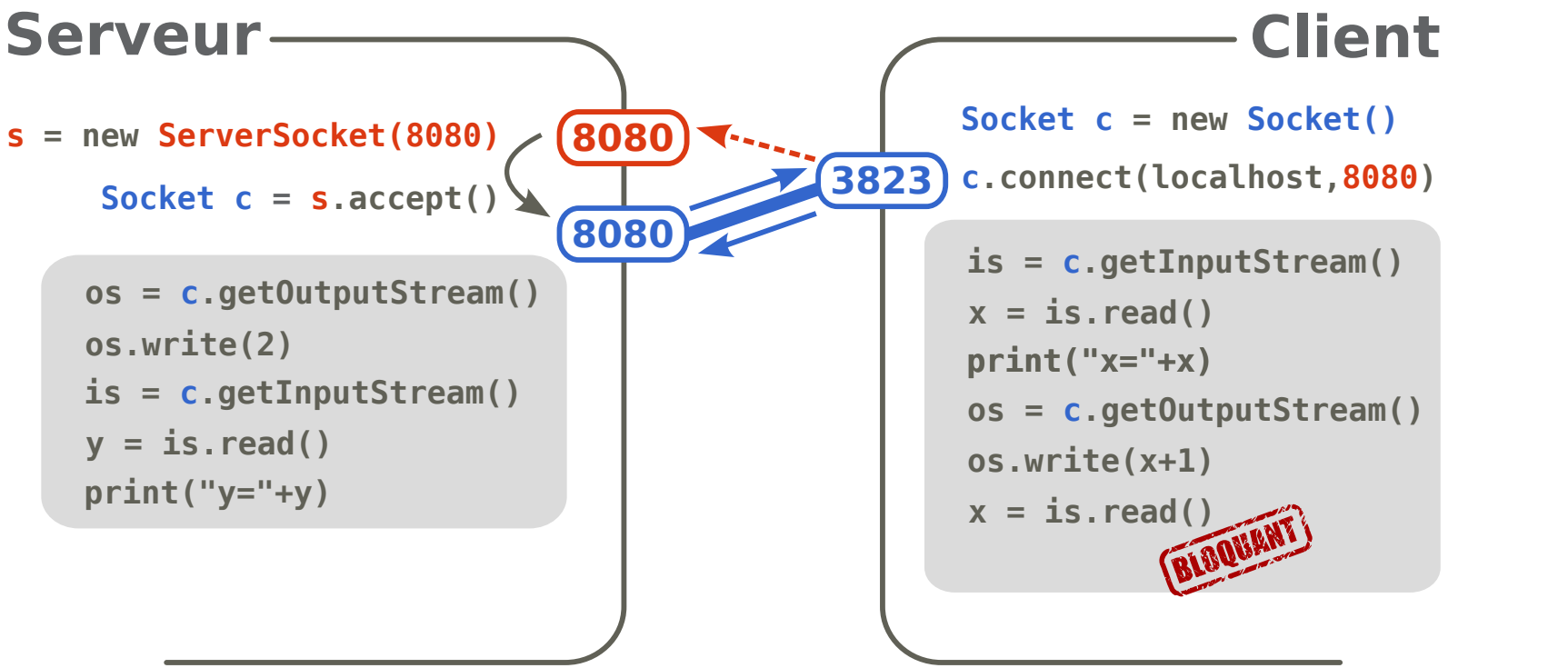
Utilisation d'une socket TCP




```
y=3
```

```
x=2
```

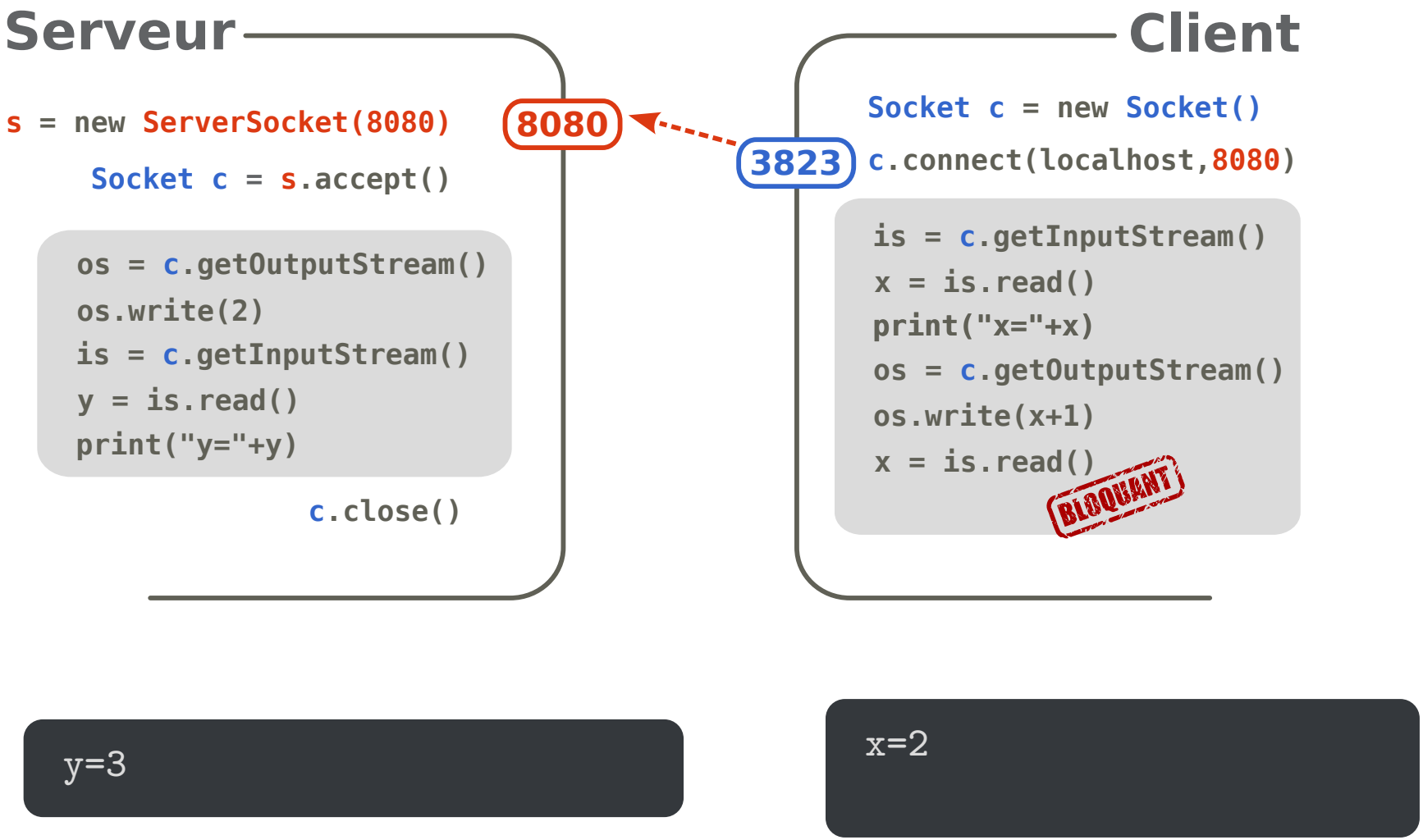
Utilisation d'une socket TCP



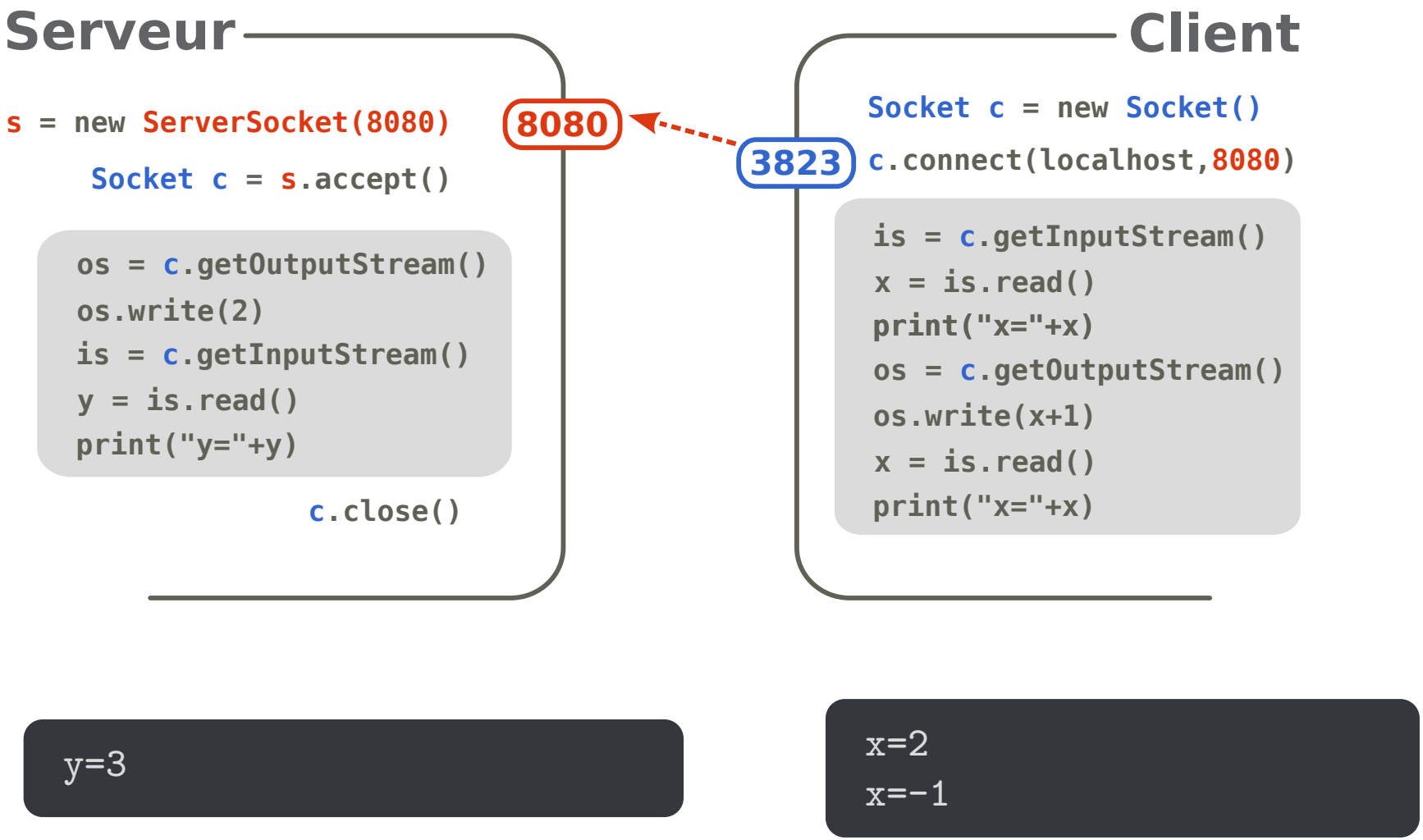

```
y=3
```

```
x=2
```

Utilisation d'une socket TCP



Utilisation d'une socket TCP



Utilisation d'une socket TCP

Serveur

```
s = new ServerSocket(8080)
```

8080

```
Socket c = s.accept()
```

```
os = c.getOutputStream()
os.write(2)
is = c.getInputStream()
y = is.read()
print("y="+y)
```

```
c.close()
```

Client

```
Socket c = new Socket()
c.connect(localhost,8080)
```

```
is = c.getInputStream()
x = is.read()
print("x="+x)
os = c.getOutputStream()
os.write(x+1)
x = is.read()
print("x="+x)
```

```
c.close()
```

```
y=3
```

```
x=2
x=-1
```

Utilisation d'une socket TCP

Serveur

```
s = new ServerSocket(8080)
     Socket c = s.accept()
os = c.getOutputStream()
os.write(2)
is = c.getInputStream()
y = is.read()
print("y="+y)
                c.close()
                s.close()
```

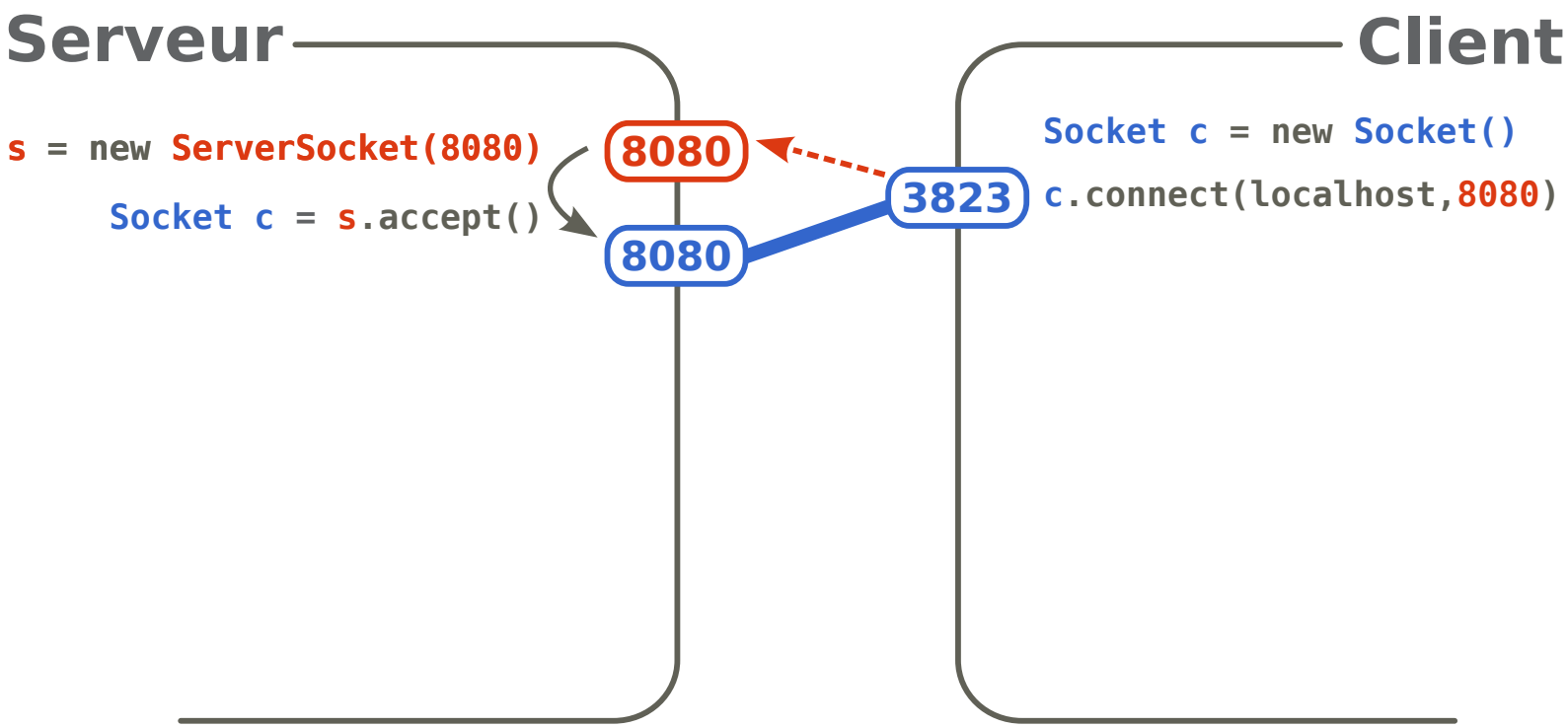
Client

```
Socket c = new Socket()
c.connect(localhost,8080)
is = c.getInputStream()
x = is.read()
print("x="+x)
os = c.getOutputStream()
os.write(x+1)
x = is.read()
print("x="+x)
c.close()
```

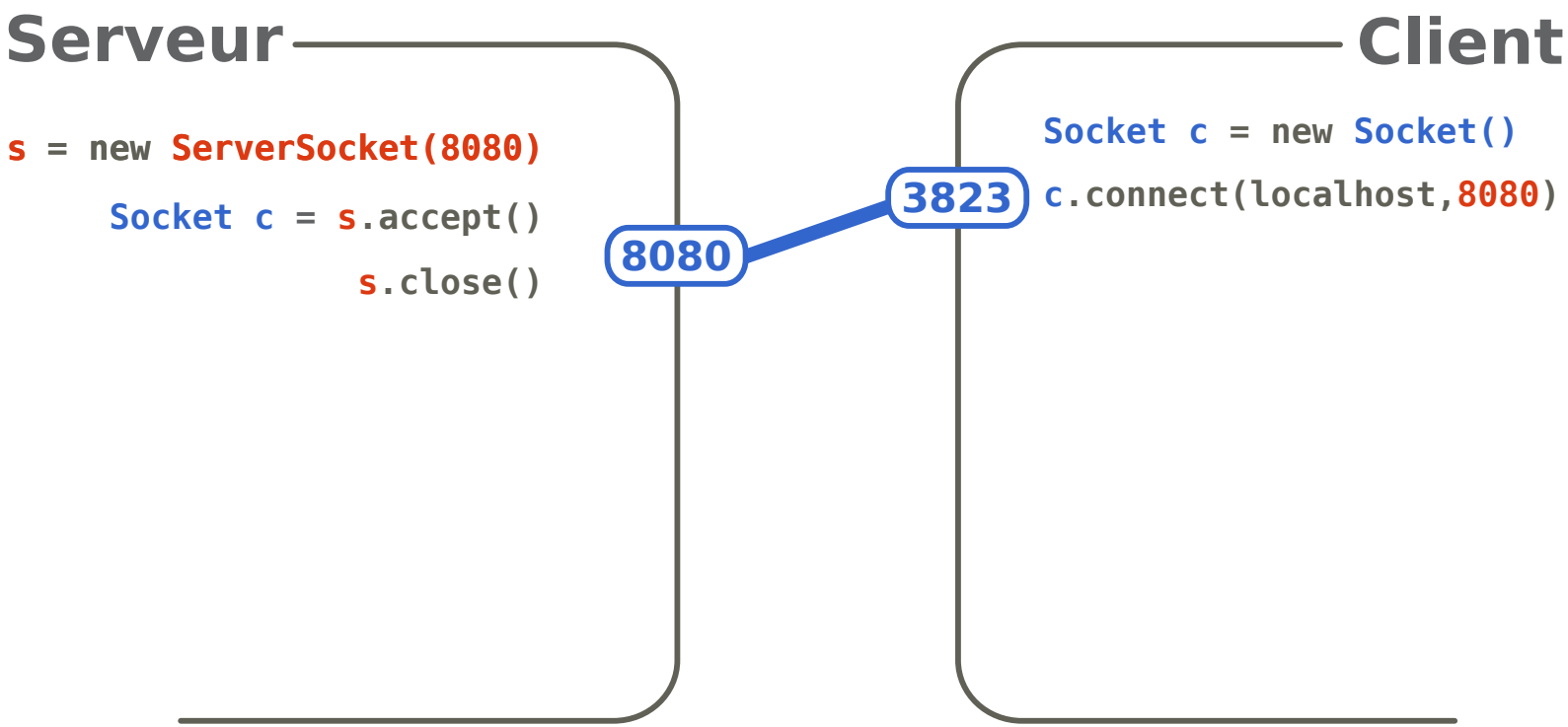
```
y=3
```

```
x=2
x=-1
```

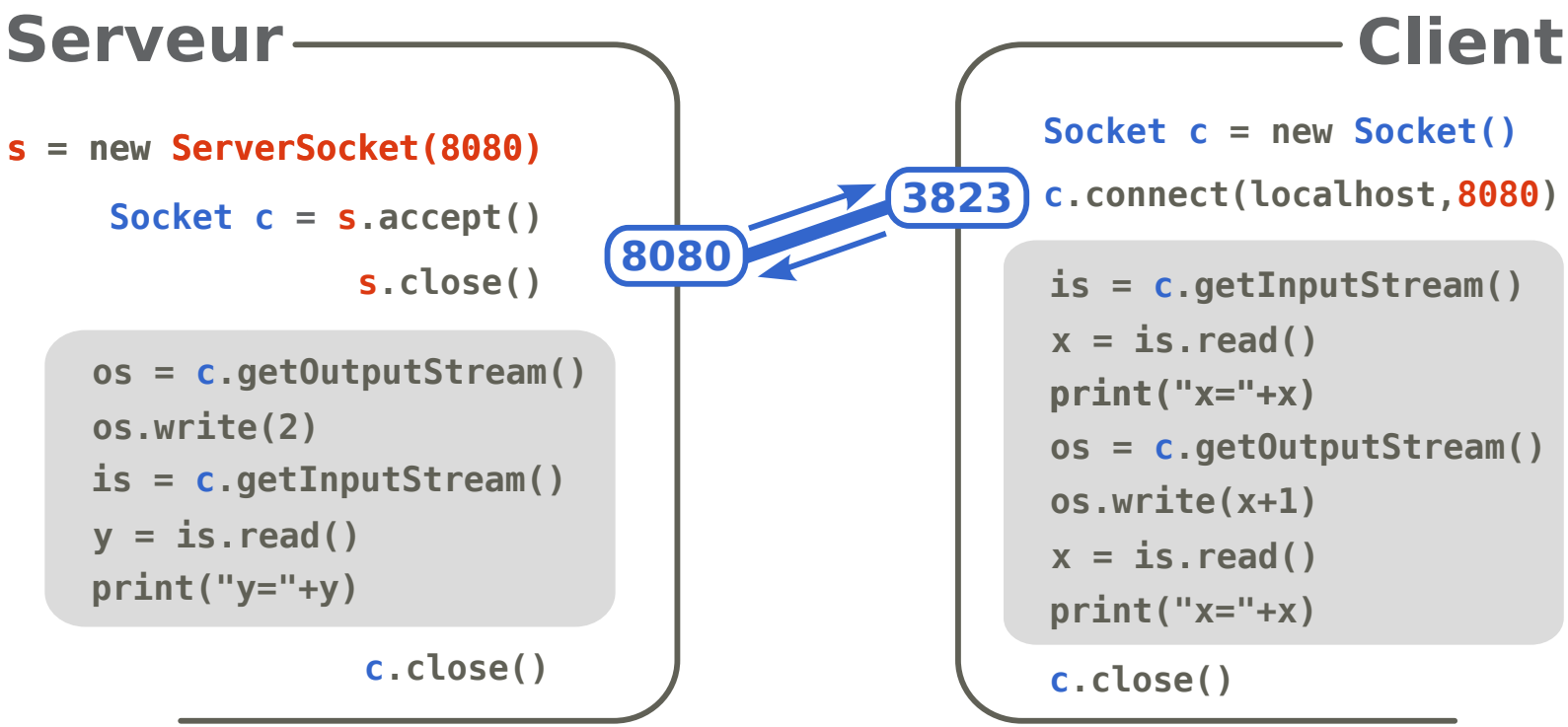
Fermeture de la socket de connexion



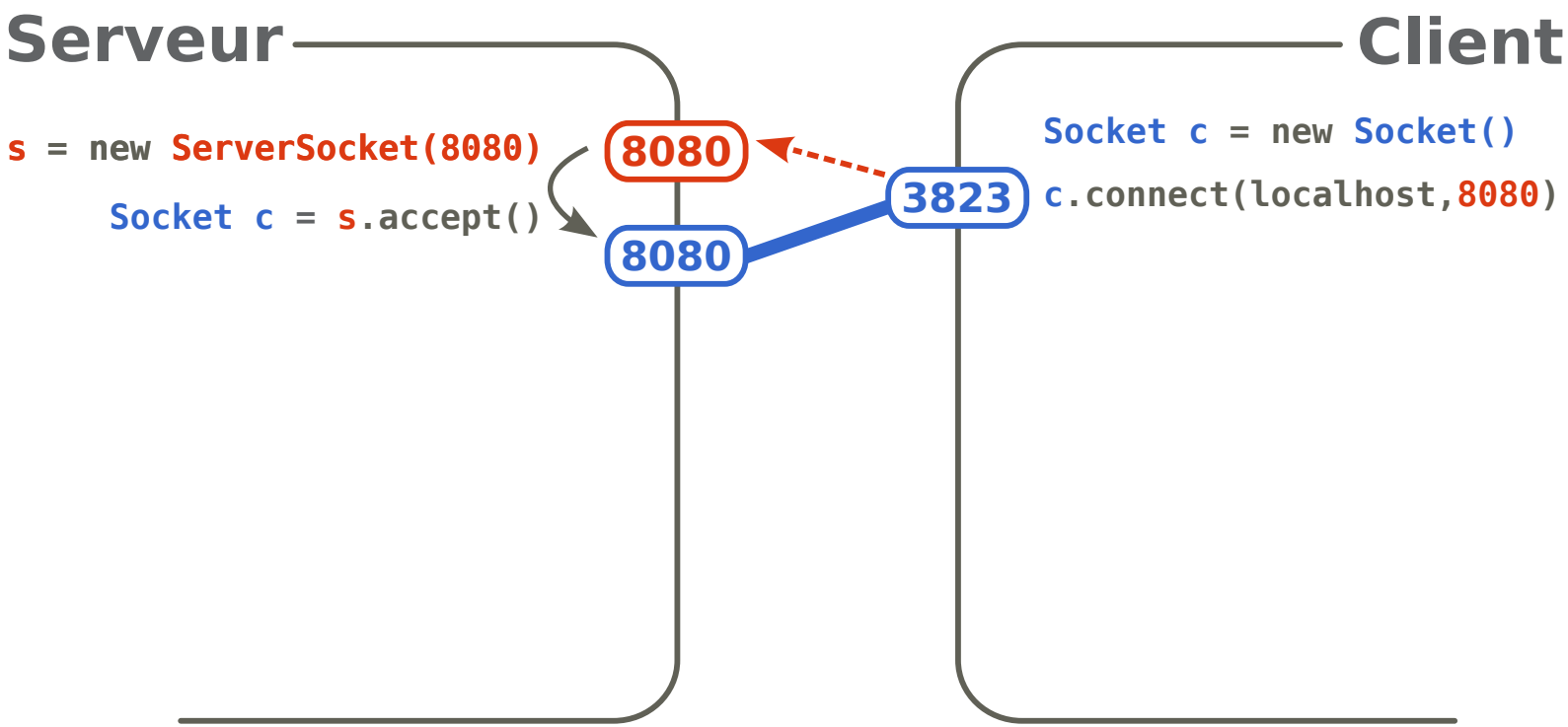
Fermeture de la socket de connexion



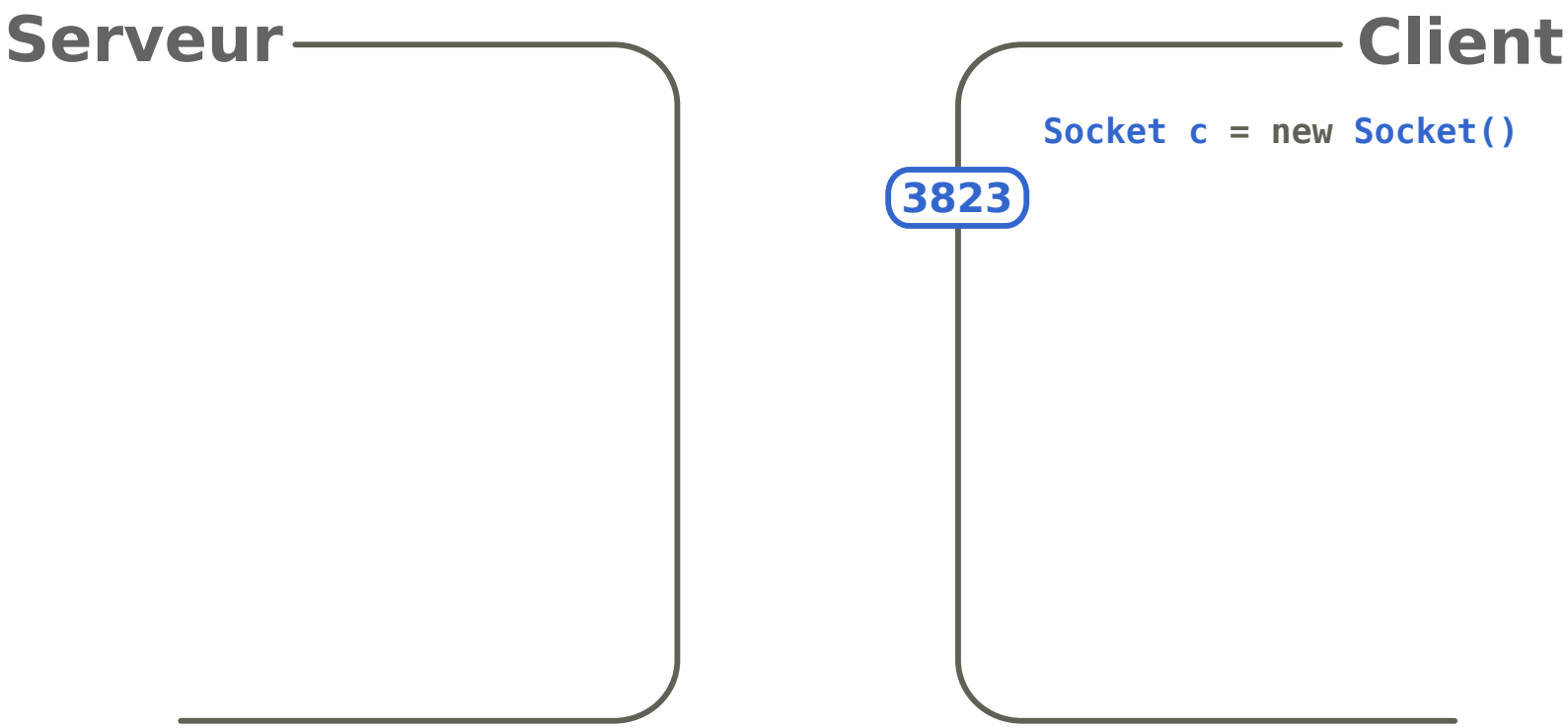
Fermeture de la socket de connexion



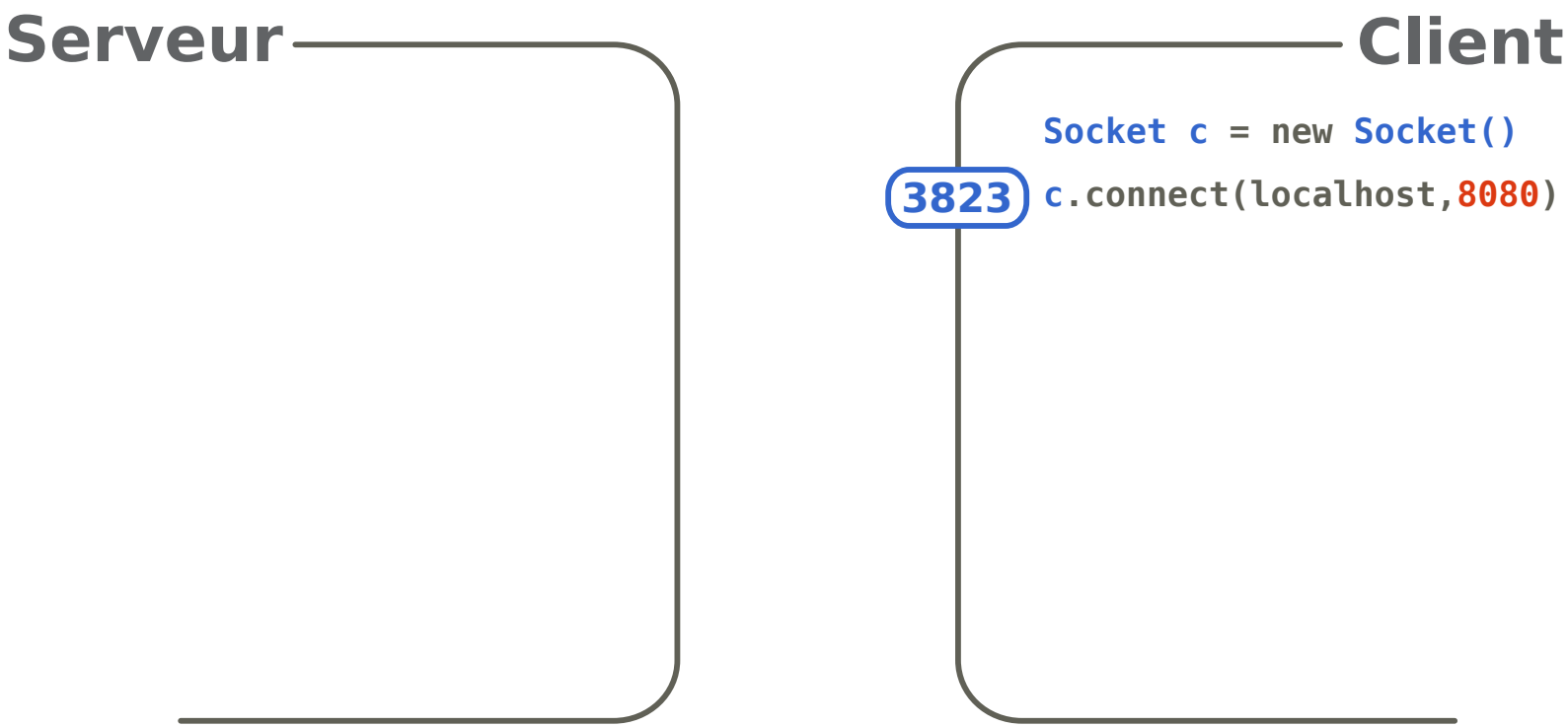
Connexion précoce : pas de socket de connexion



Connexion précoce : pas de socket de connexion



Connexion précoce : pas de socket de connexion



Connexion précoce : pas de socket de connexion

Serveur

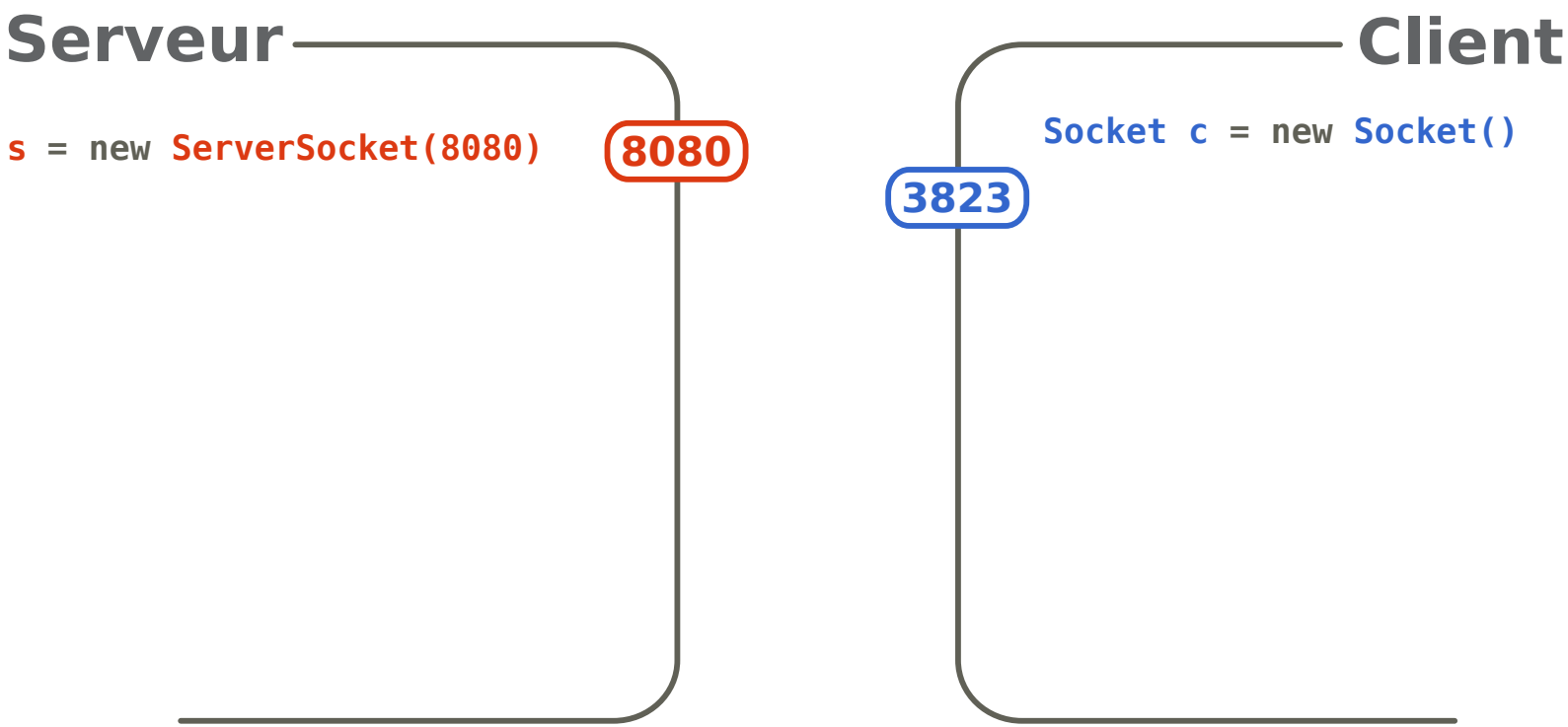
Client

```
Socket c = new Socket()
c.connect(localhost,8080)
```

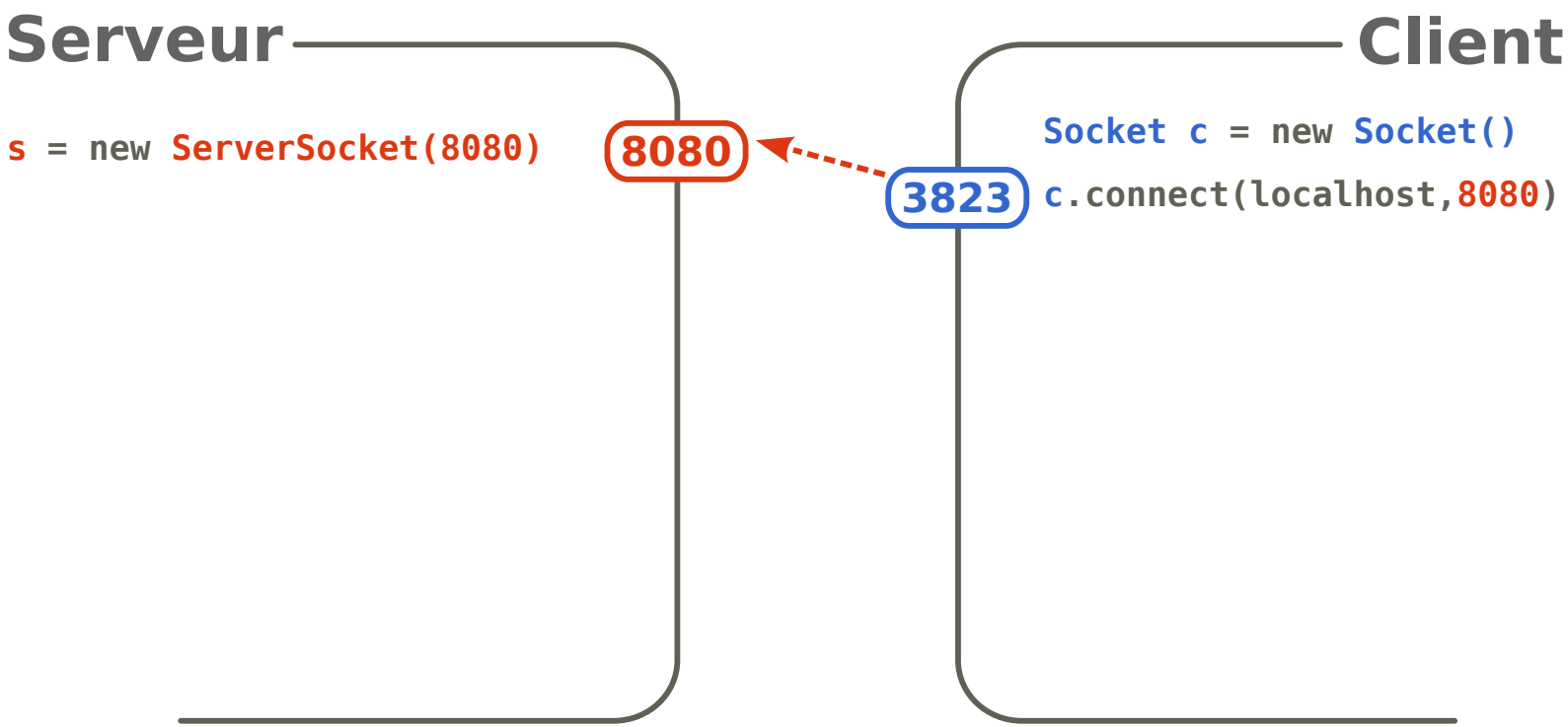
3823

ConnectException

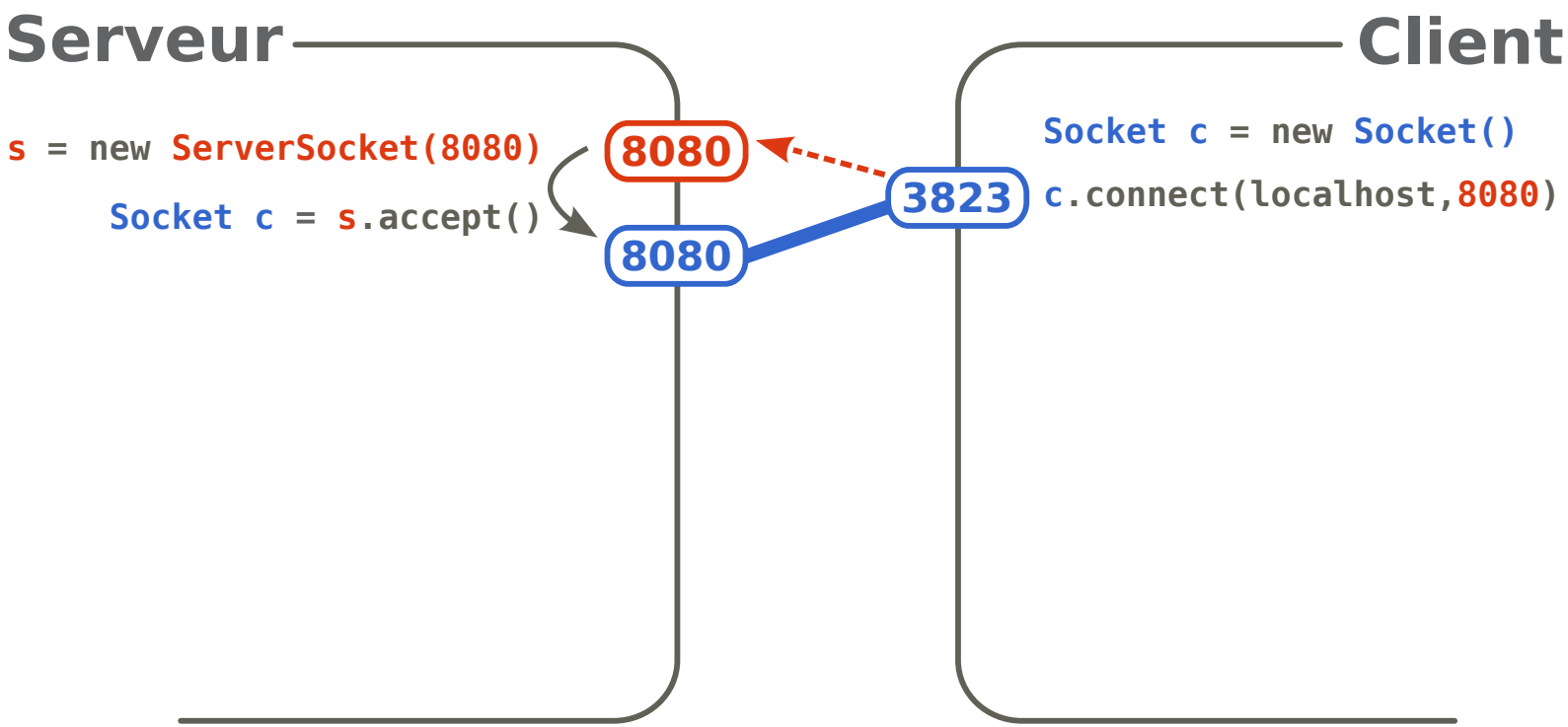
Connexion précoce : avant le accept du serveur



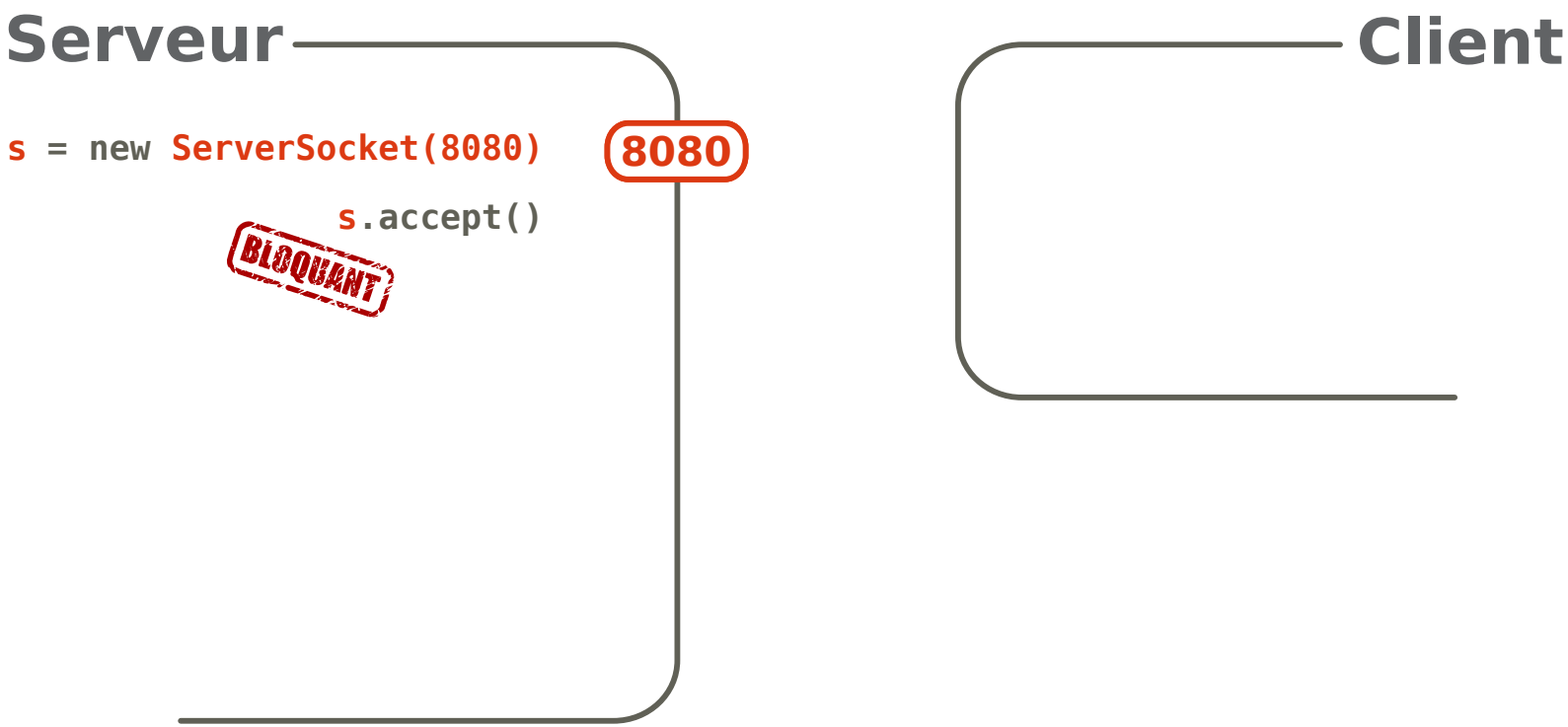
Connexion précoce : avant le accept du serveur



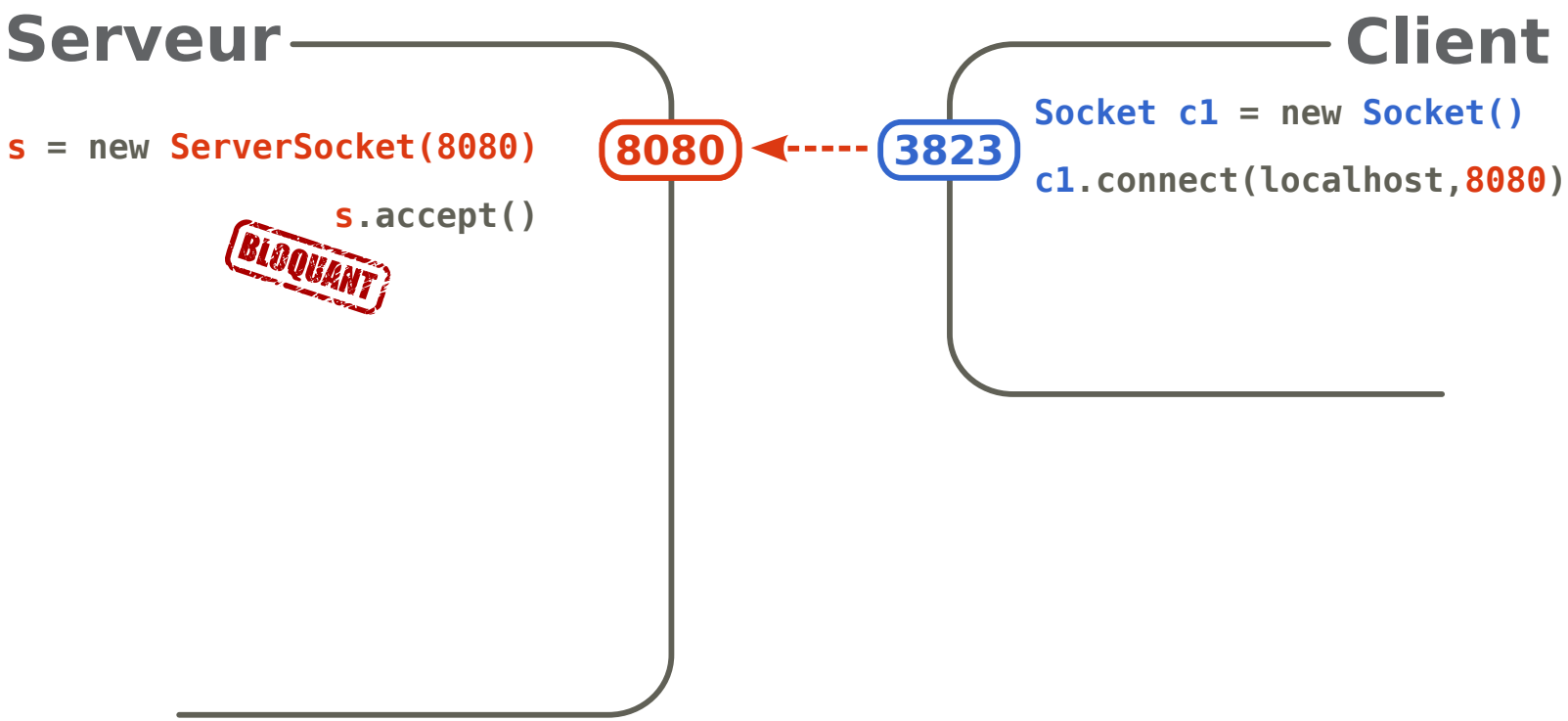
Connexion précoce : avant le accept du serveur



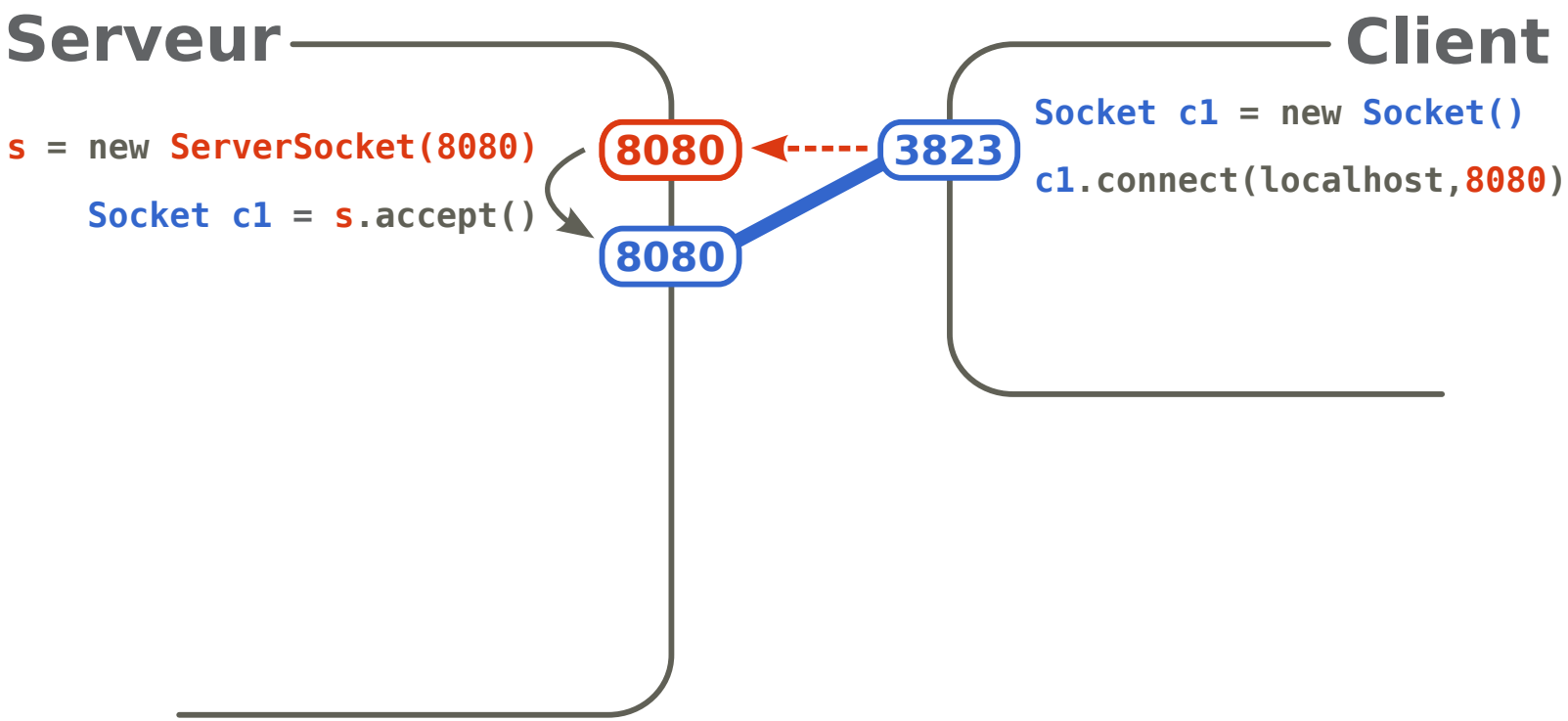
Connexions multiples sur une socket TCP



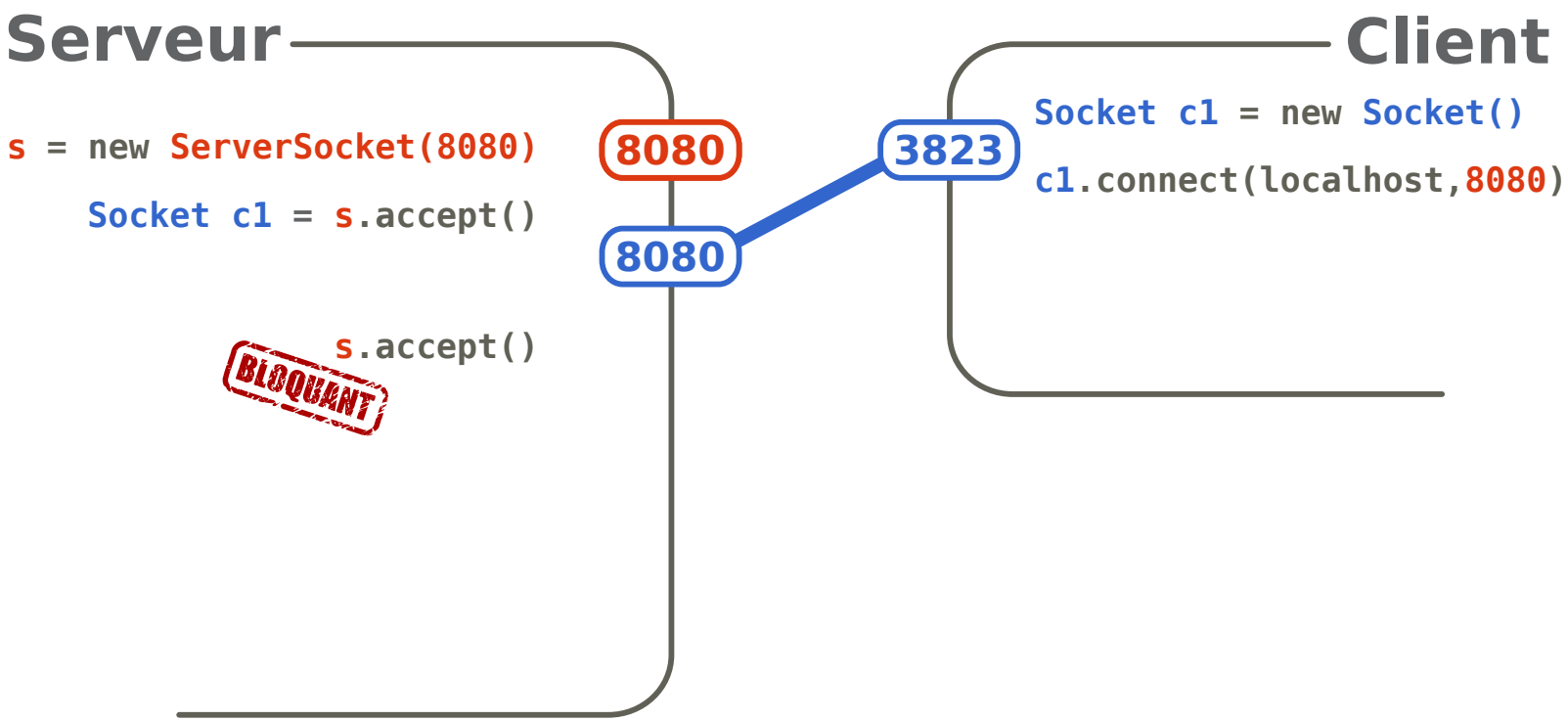
Connexions multiples sur une socket TCP



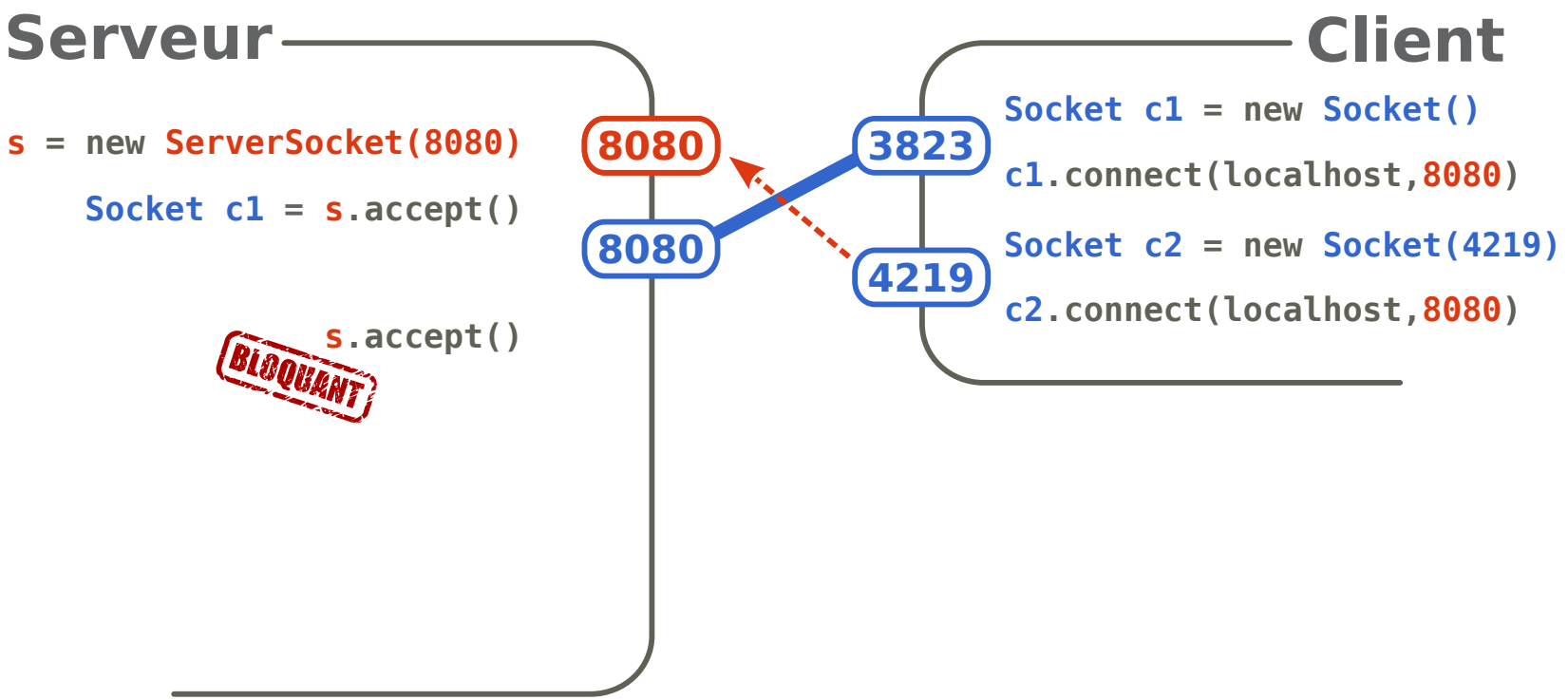
Connexions multiples sur une socket TCP



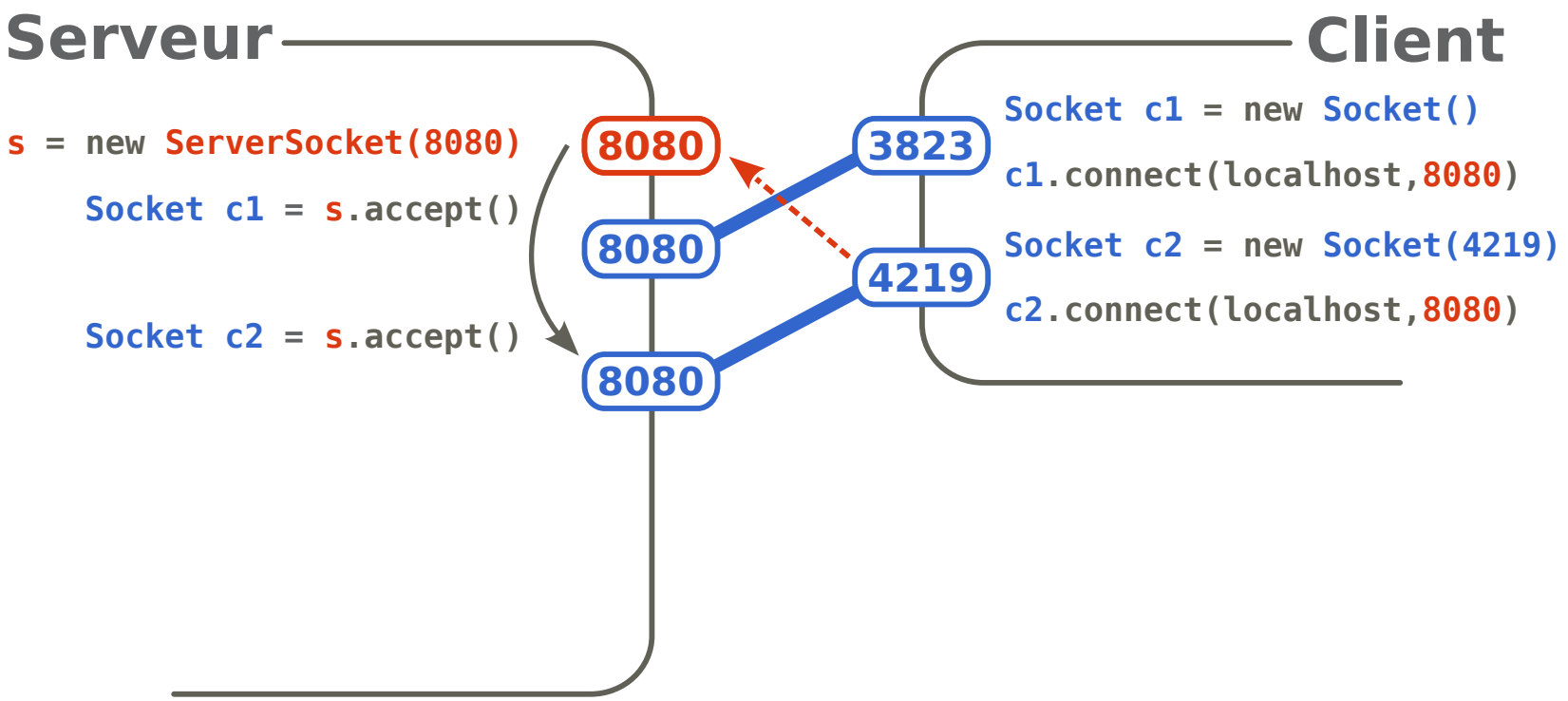
Connexions multiples sur une socket TCP



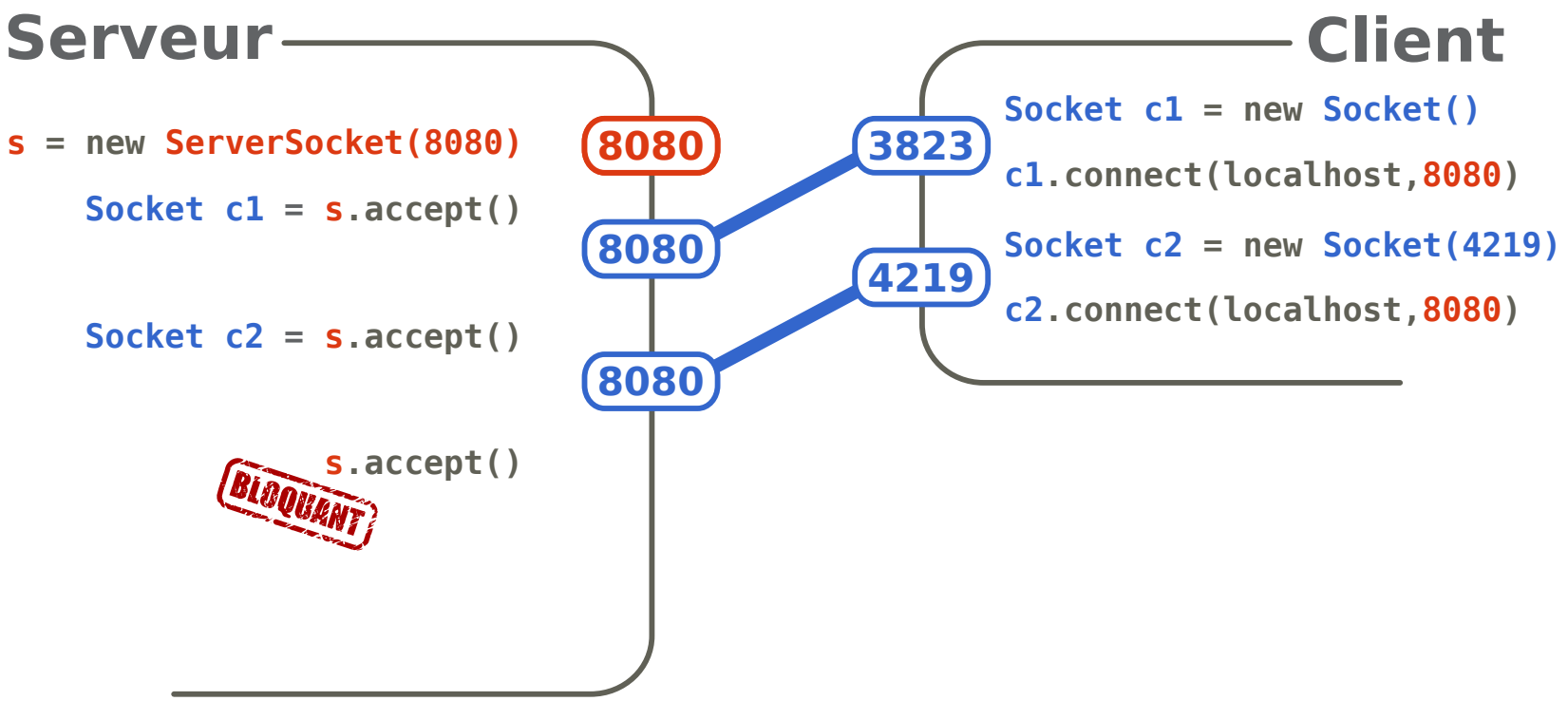
Connexions multiples sur une socket TCP



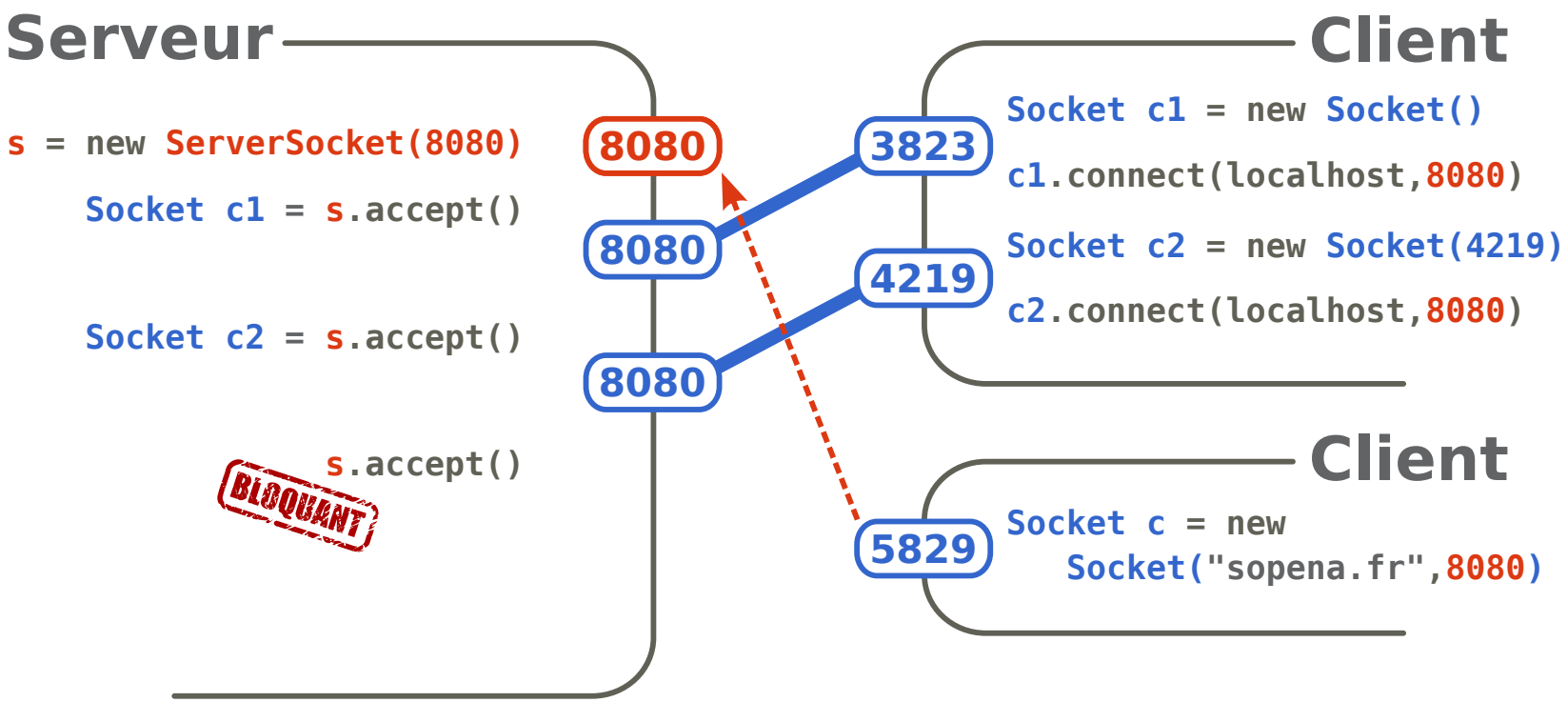
Connexions multiples sur une socket TCP



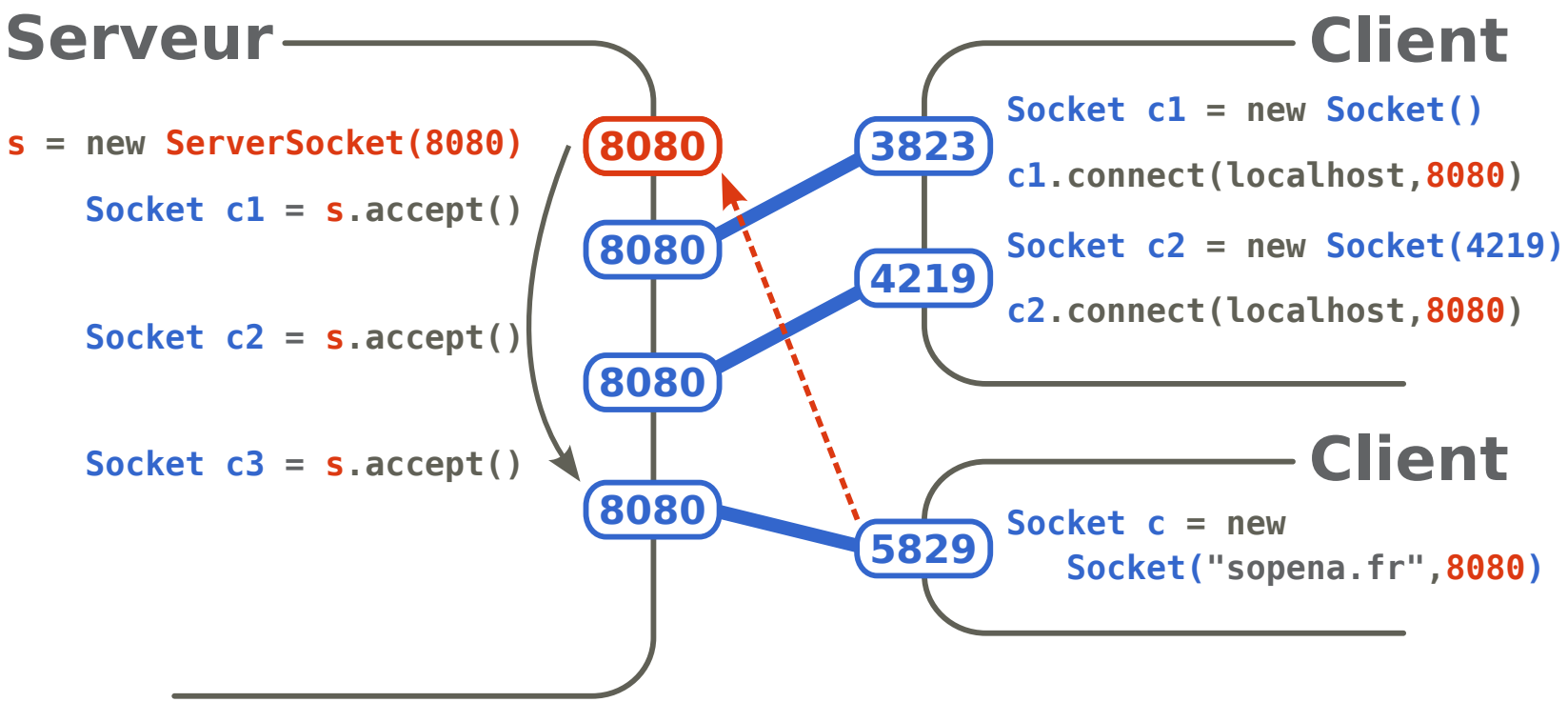
Connexions multiples sur une socket TCP



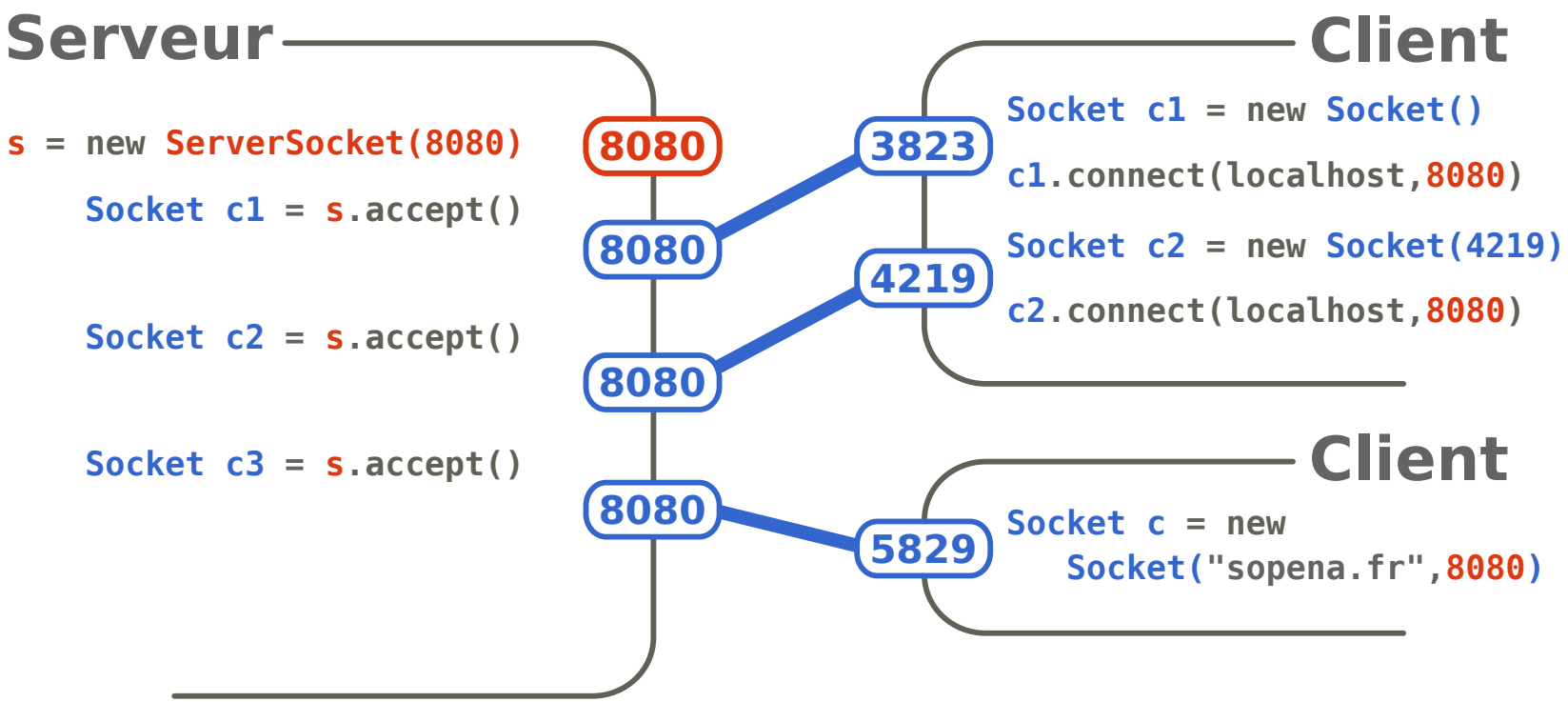
Connexions multiples sur une socket TCP



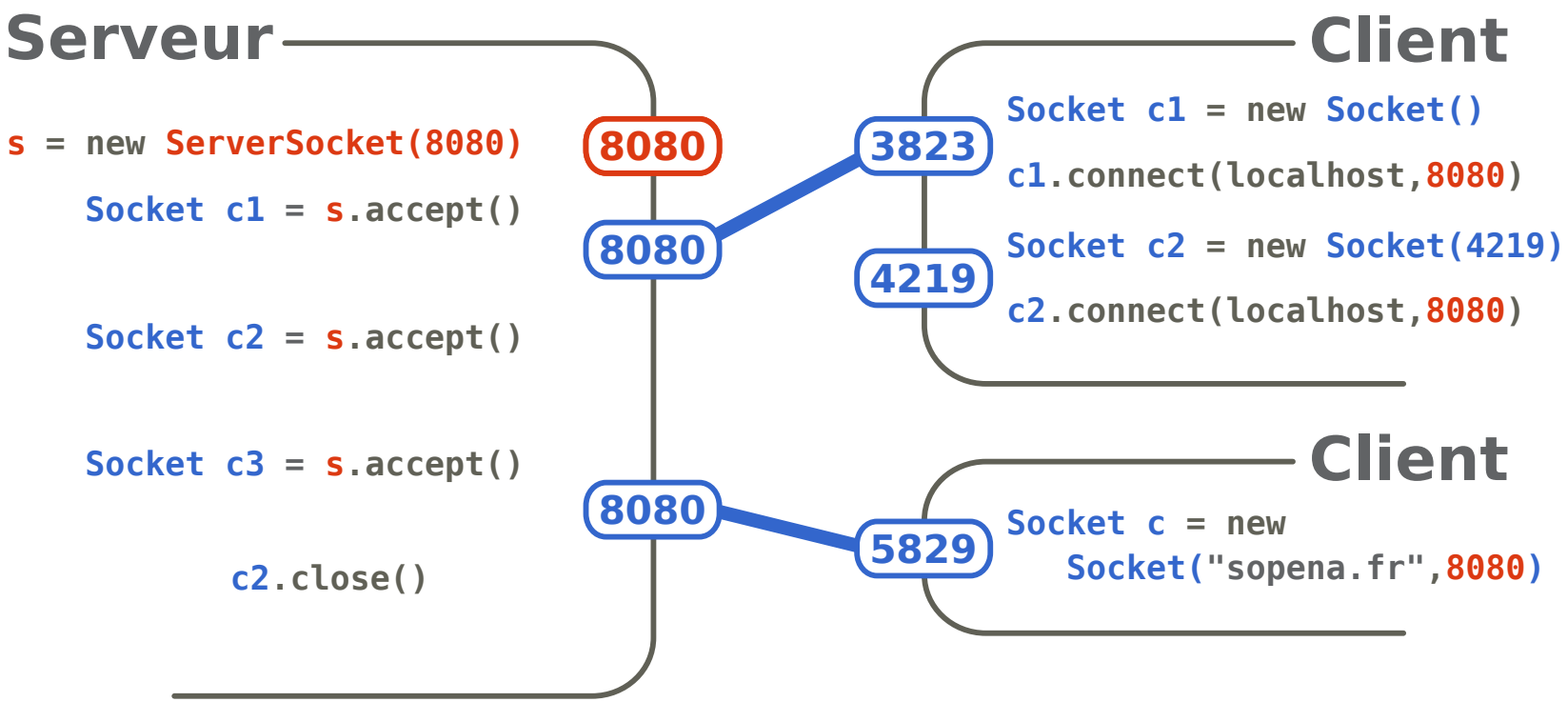
Connexions multiples sur une socket TCP



Connexions multiples sur une socket TCP

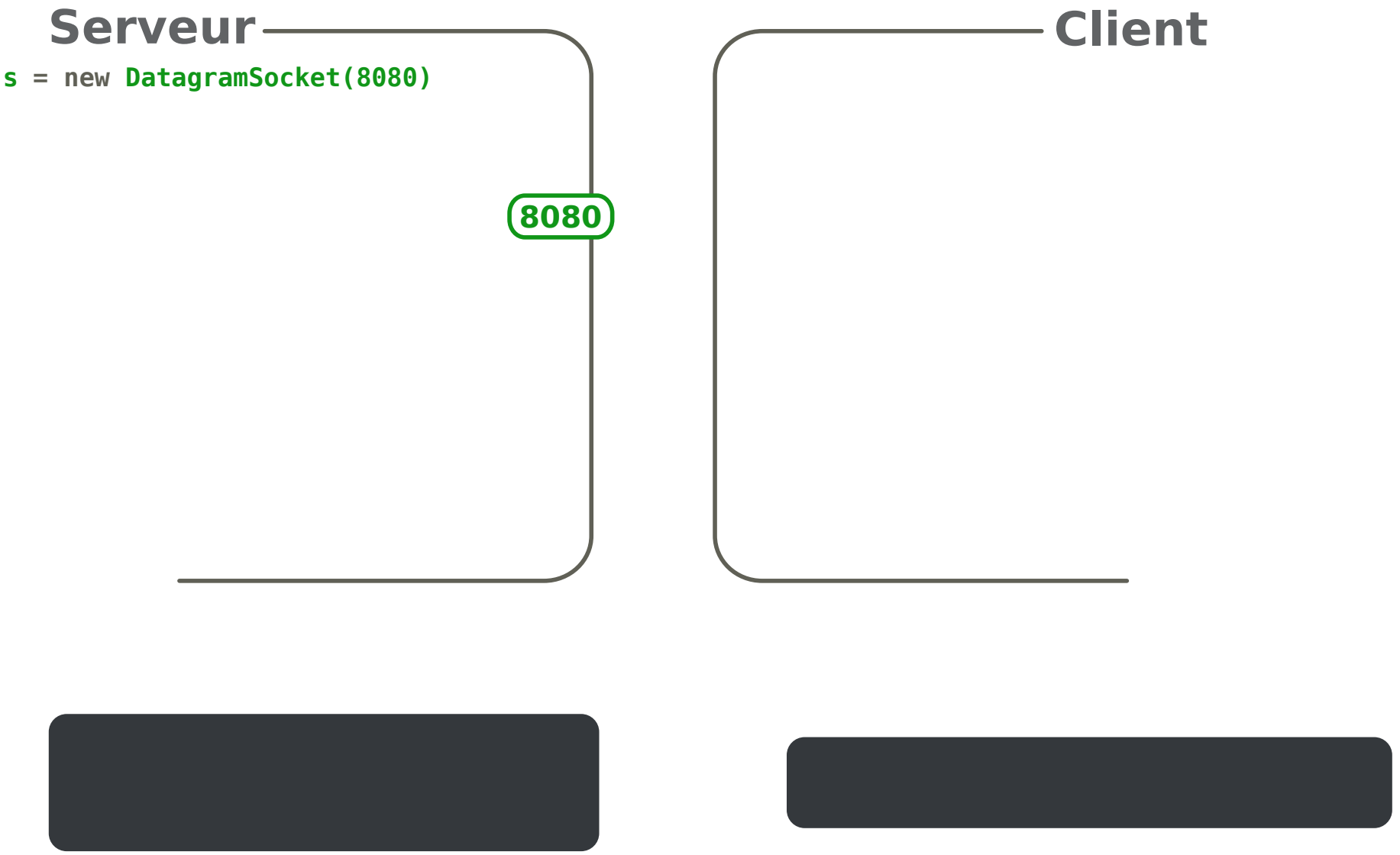


Connexions multiples sur une socket TCP



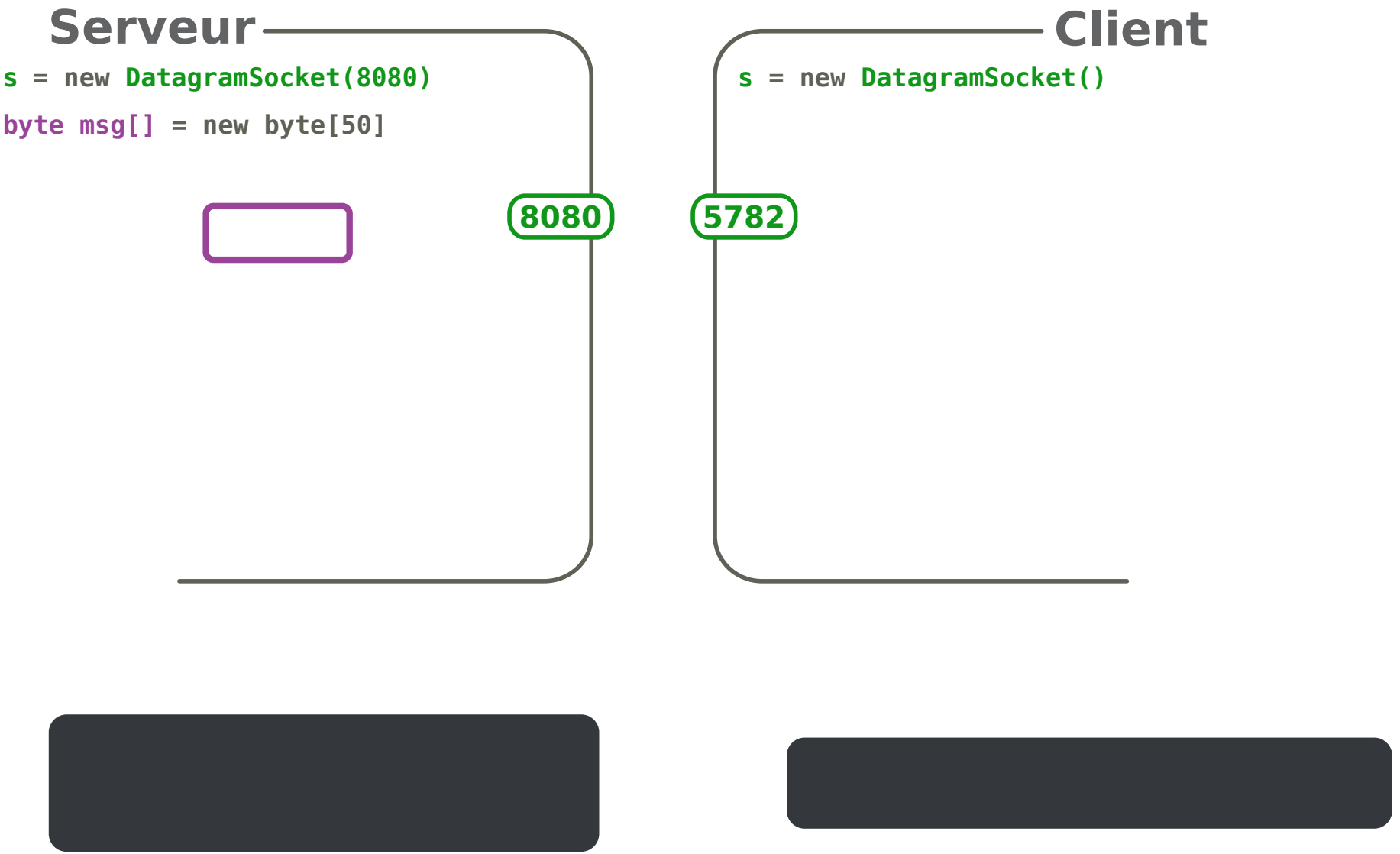
Utilisation d'une socket UDP

Utilisation d'une socket UDP



Utilisation d'une socket UDP

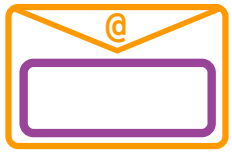
Utilisation d'une socket UDP



Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```




```
DatagramPacket in =
          new DatagramPacket(msg, 50)
```

8080

Client

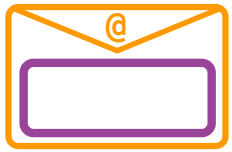
```
s = new DatagramSocket()
```

5782

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```




```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

8080

Client

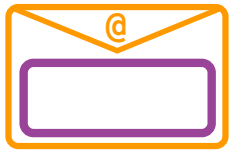
```
s = new DatagramSocket()
```

5782

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```



8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

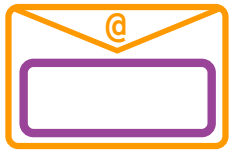
```
s = new DatagramSocket()
```

5782

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```



8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

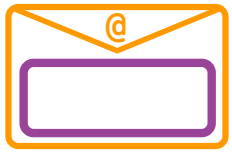
5782

Hello!

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```



8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

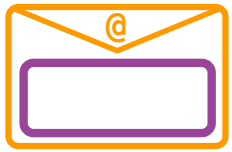
5782


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
```

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```




```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
```

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
```

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
System.out.println(new String(msg))
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
```

8080 ← 5782

```
Hello !
```

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
System.out.println(new String(msg))
System.out.println("From port : " +
                        in.getPort())
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
```

```
Hello !
From port : 5782
```

Utilisation d'une socket UDP

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
System.out.println(new String(msg))
System.out.println("From port : " +
                        in.getPort())
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
System.out.println("To port : " +
                        out.getPort())
```

```
Hello !
From port : 5782
```

```
To port : 8080
```

Émission précoce : avant ouverture de l'autre socket

Émission précoce : avant ouverture de l'autre socket

Émission précoce : avant ouverture de l'autre socket

Serveur

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

5782

Hello!

Émission précoce : avant ouverture de l'autre socket

Serveur

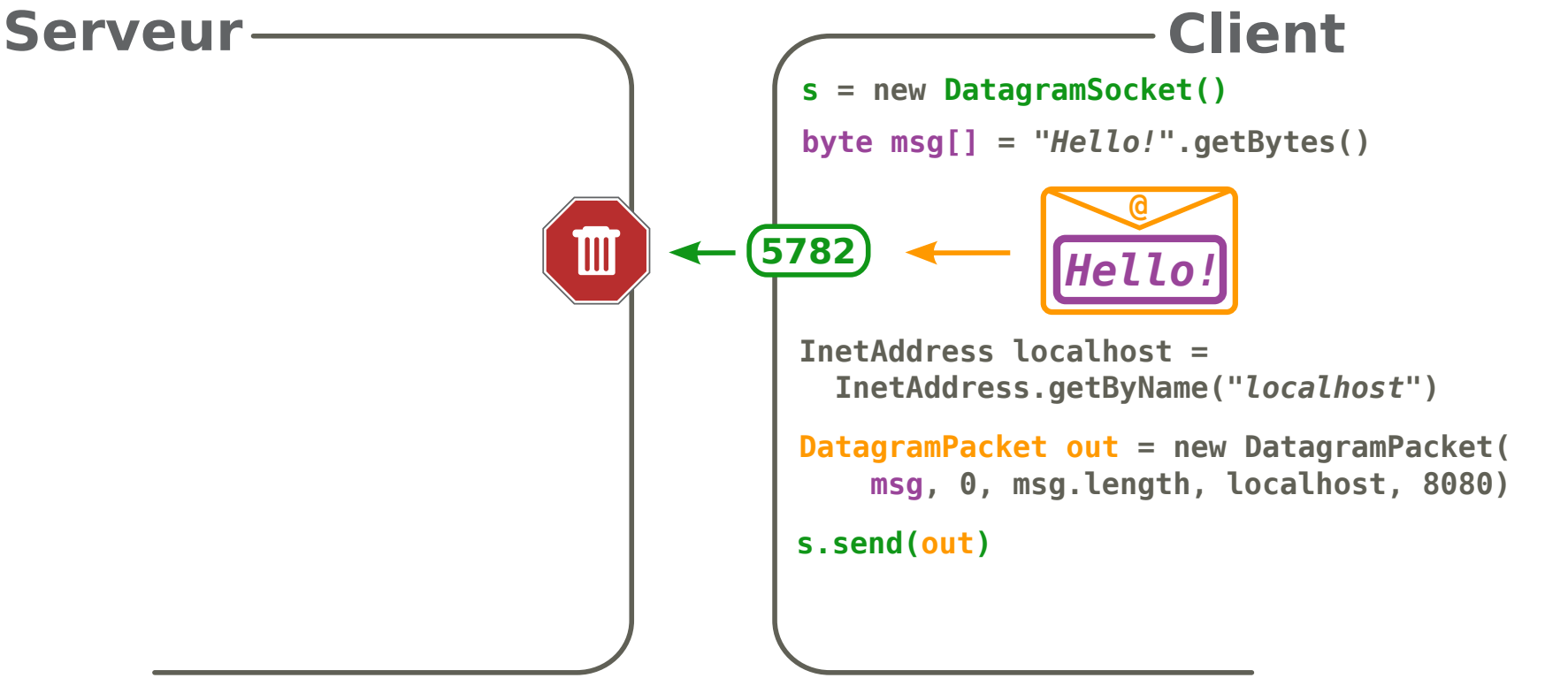
Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

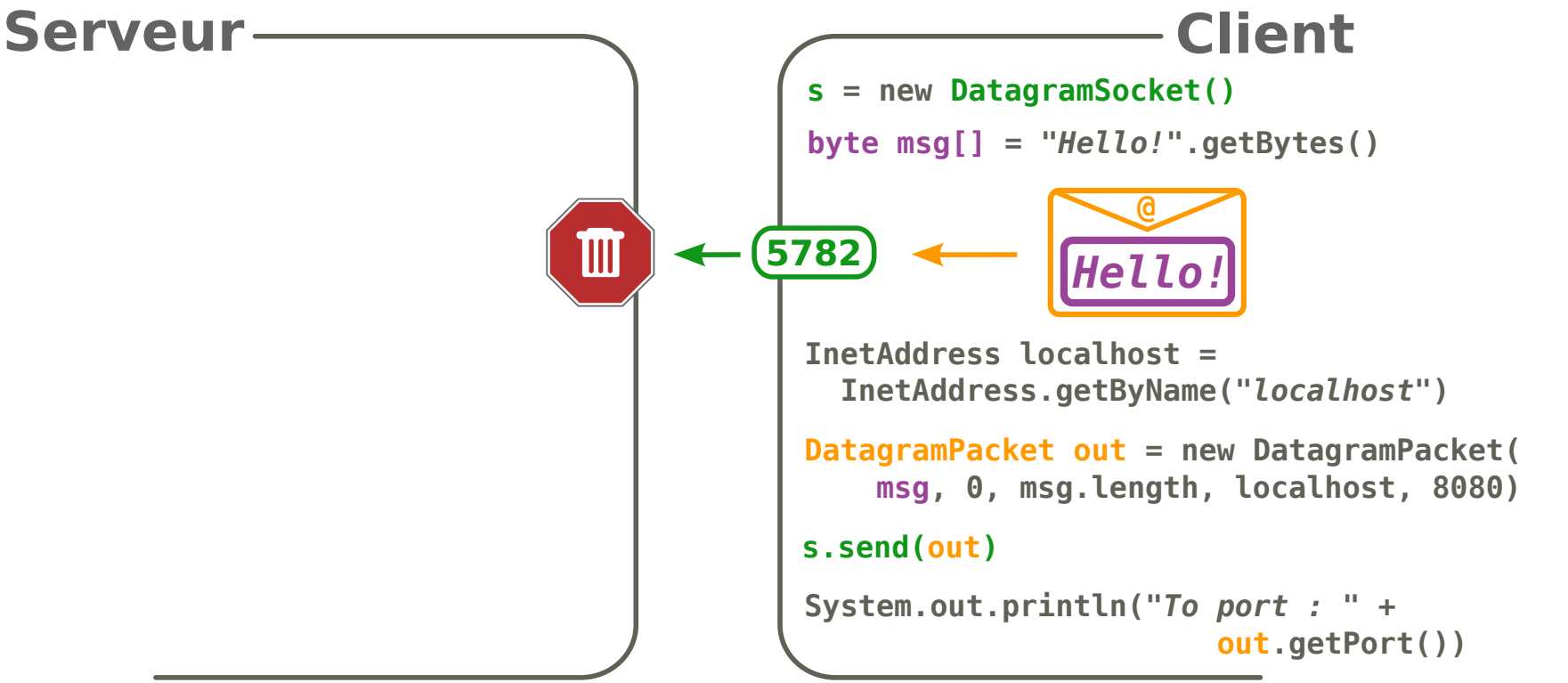
5782


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
```

Émission précoce : avant ouverture de l'autre socket



Émission précoce : avant ouverture de l'autre socket

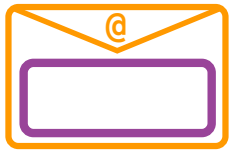



```
To port : 8080
```

Émission précoce : avant ouverture de l'autre socket

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```



```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

BLOQUANT

8080

5782

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
System.out.println("To port : " +
                        out.getPort())
```

```
To port : 8080
```

Émission précoce : avant le `receive` du serveur

Émission précoce : avant le `receice` du serveur

Serveur

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

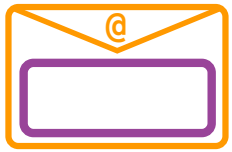
5782


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
```

Émission précoce : avant le `receice` du serveur

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```



8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

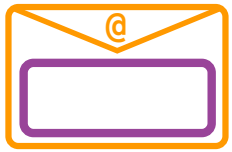
5782


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
```

Émission précoce : avant le `receice` du serveur

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```




```
DatagramPacket in =
          new DatagramPacket(msg, 50)
```

8080 ← 5782

Client

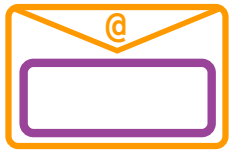
```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
```

Émission précoce : avant le `receice` du serveur

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```




```
DatagramPacket in =
          new DatagramPacket(msg, 50)
```

8080 ← 5782

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
System.out.println("To port : " +
                        out.getPort())
```

```
To port : 8080
```

Émission précoce : avant le `receice` du serveur

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

8080 ← 5782

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
System.out.println("To port : " +
                        out.getPort())
```

```
To port : 8080
```

Émission précoce : avant le `receice` du serveur

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
System.out.println(new String(msg))
System.out.println("From port : " +
                        in.getPort())
```

8080 ← 5782

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
System.out.println("To port : " +
                        out.getPort())
```

```
Hello !
From port : 5782
```

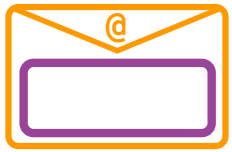
```
To port : 8080
```

Buffer de réception trop petit

Buffer de réception trop petit

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[50]
```



8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

Buffer de réception trop petit

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[4]
```

8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

BLOQUANT

Client

Buffer de réception trop petit

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[4]
```

8080

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```

5782


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
```

Buffer de réception trop petit

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[4]
```

```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
```

BLOQUANT

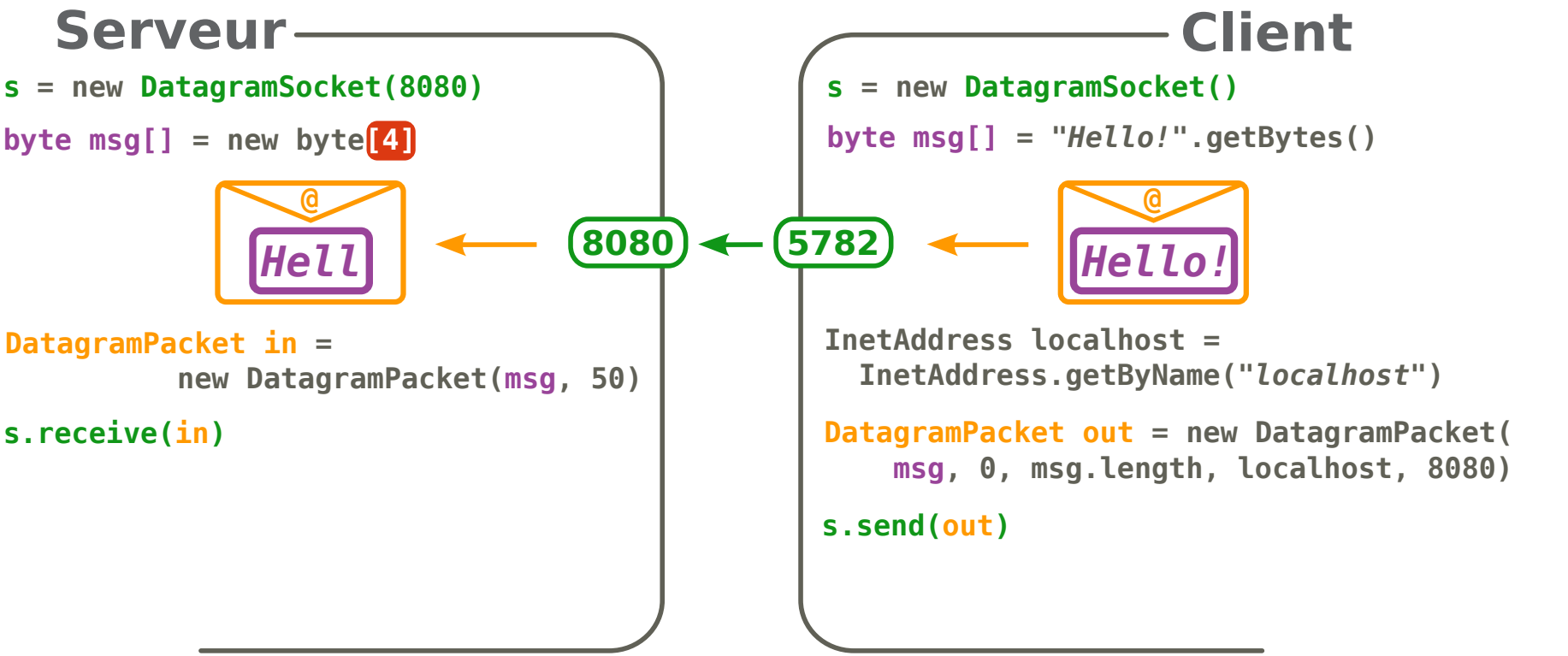
8080 ← 5782 ←

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
```

Buffer de réception trop petit



Buffer de réception trop petit

Serveur

```
s = new DatagramSocket(8080)
byte msg[] = new byte[4]
```


```
DatagramPacket in =
          new DatagramPacket(msg, 50)
s.receive(in)
System.out.println(new String(msg))
System.out.println("From port : " +
                        in.getPort())
```

8080 ← 5782

Client

```
s = new DatagramSocket()
byte msg[] = "Hello!".getBytes()
```


```
InetAddress localhost =
  InetAddress.getByName("localhost")
DatagramPacket out = new DatagramPacket(
    msg, 0, msg.length, localhost, 8080)
s.send(out)
System.out.println("To port : " +
                        out.getPort())
```

```
Hell
From port : 5782
```

```
To port : 8080
```